TÓM TẮT

Tên đề tài: Thiết kế hệ thống hỗ trợ tự lái dựa trên nhận dạng hình ảnh thời gian thực.
Sinh viên thực hiện: Ngô Đức Luỹ
Số thẻ SV: 106210193 Lớp: 21DTCLC4

LỜI NÓI ĐẦU

Trong bối cảnh cuộc Cách mạng công nghiệp 4.0 đang diễn ra mạnh mẽ trên toàn cầu, ngành công nghiệp ô tô tự hành đã và đang trở thành một trong những lĩnh vực tiên phong, hứa hẹn mang lại những thay đổi căn bản cho hệ thống giao thông — hướng tới mục tiêu an toàn tuyệt đối, hiệu quả và thân thiện với môi trường.

Các tập đoàn công nghệ và sản xuất ô tô hàng đầu thế giới như Tesla, Waymo, Cruise đang không ngừng nghiên cứu và cạnh tranh để biến giấc mơ về những chiếc xe hoàn toàn tự hành trở thành hiện thực. Tại Việt Nam, cùng với sự phát triển của các hệ thống giao thông thông minh (ITS), những nghiên cứu đầu tiên về phương tiện tự động và hỗ trợ lái xe thông minh cũng đã bắt đầu được triển khai, mở ra hướng đi đầy tiềm năng cho nền kỹ thuật trong nước.

Xuất phát từ nhận thức đó, cùng với niềm đam mê nghiên cứu khoa học và mong muốn được đóng góp một phần vào sự phát triển chung của lĩnh vực này, em đã lựa chọn đề tài "Hệ thống hỗ trợ lái xe dựa trên nhận dạng hình ảnh thời gian thực" làm nội dung nghiên cứu cho đồ án môn học của mình.

Trong quá trình thực hiện đề tài, em xin gửi lời cảm ơn chân thành đến thầy Tăng Anh Tuấn, giảng viên Khoa Điện Tử – Viễn Thông, Trường Đại học Bách Khoa – Đại học Đà Nẵng, người đã tận tình hướng dẫn, chỉ bảo và động viên em trong suốt quá trình thực hiện.

Mặc dù đã cố gắng hoàn thiện đề tài trong khả năng của mình, nhưng do kiến thức và kinh nghiệm thực tế còn hạn chế, chắc chắn đồ án không tránh khỏi những thiếu sót. Em rất mong nhận được ý kiến đóng góp và sự chỉ bảo của quý thầy cô để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn!

LỜI CAM KẾT

Sinh viên thực hiện đề tài, cam đoan rằng đồ án này được thực hiện với sự trung thực, không sao chép đề tài của người khác và tuân thủ các quy định về liêm chính học thuật. Mọi sự giúp đỡ cho việc thực hiện bản đề tài này đã được trích dẫn trong bản đồ án đã được chỉ rõ, nguồn gốc rõ ràng.

Ký tên

Sinh viên thực hiện

Ngô Đức Luỹ

DANH SÁCH HÌNH ẢNH VÀ BẢNG BIỂU

1. Danh sách hình

Hình 2.1	Ví dụ về hệ thống ADAS trong thực tiễn
Hình 2.2	Hệ thống LDW trong thực tiễn
Hình 2.3	Hệ thống nhận diện vật thể
Hình 2.4	Hệ thống phanh khẩn cấp tự động
Hình 2.5	Chuyển đổi không gian màu
Hình 2.6	Phép làm mờ ảnh
Hình 2.7	Phát hiện cạnh
Hình 2.8	Phép biến đổi góc nhìn
Hình 2.9	Cấu trúc CNN cơ bản
Hình 2.10	Nguyên lý hoạt động của YOLO
Hình 3.1	Lưu đồ thuật toán của mô hình nhận diện vật thể
Hình 3.2	Lưu đồ thuật toán của mô hình nhận diện làn đường
Hình 3.3	Các điểm landmarks được xác định trên mặt
Hình 3.4	Lưu đồ thuật toán của mô hình nhận diện trạng thái mắt
Hình 3.5	Cấu trúc phần cứng khi không sử dụng đa tiến trình
Hình 3.6	Cấu trúc phần cứng khi sử dụng đa tiến trình
Hình 3.7	Kiến trúc thiết kế cho hệ thống tổng thể
Hình 4.1	Kết quả thử nghiệm của mô hình nhận diện vật thể
Hình 4.2	Kết quả của mô hình khi dùng Yolov3
Hình 4.3	Hiện tượng vật thể bị nhận diện nhiều lần
Hình 4.4	FPS thực tế được đo ở chế độ inference-only
Hình 4.5	Kết quả nhận diện làn đường
Hình 4.6	Mô hình hoạt động không tốt trong môi trường nhiều phương tiện

Hình 4.7	FPS của mô hình nhận diện làn đường ở chế độ inference-only
Hình 4.8	Trường hợp không nhận biết được khuôn mặt
Hình 4.9	Trường hợp nhắm mắt và mở mắt
Hình 4.10	Độ tương phản làm ảnh hưởng khả năng dự đoán của mô hình
Hình 4.11	Kết quả FPS thực tế
Hình 4.12	Kết quả khi chạy một lần ba mô hình nhận diện
Hình 5.1	NVIDIA AI Jetson Nano
Hình 5.2	Raspberry Pi Camera Module V2 8MP
Hình 5.3	Cảm biến siêu âm HC-R704
Hình 5.4	GY-9250 9DOF IMU MPU9250
Hình 5.5	Cảm biến tốc độ ELM327 USB OBD2

2. Danh sách bảng

Bảng 3.1	Mối quan hệ giữa gia tốc, quãng đường giảm tốc và thời gian
Bảng 3.2	Các tham số trong không gian HSV
Bảng 3.3	So sánh sử dụng Process và Thread
Bảng 4.1	Môi trường thực nghiệm
Bảng 4.2	Đánh giá hiệu năng tổng thể của hệ thống

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ	Ý nghĩa
ADAS	Advanced Driver Assistance System	Hệ thống hỗ trợ lái xe nâng cao
AI	Artificial Intelligence	Trí tuệ nhân tạo
CPU	Central Processing Unit	Bộ xử lý trung tâm
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
YOLO	You Only Look Once	Thuật toán nhận diện đối tượng thời gian thực
FPS	Frames Per Second	Số khung hình xử lý trong một giây
LDW	Lane Departure Warning	Cảnh báo lệch làn đường
FCW	Forward Collision Warning	Cảnh báo va chạm phía trước
DMS	Driver Monitoring System	Hệ thống giám sát trạng thái người lái
EAR	Eye Aspect Ratio	Chỉ số tỉ lệ mắt dùng phát hiện buồn ngủ
ROI	Region of Interest	Vùng quan tâm trong ảnh
IMU	Inertial Measurement Unit	Cảm biến quán tính
CSI	Camera Serial Interface	Chuẩn giao tiếp camera
API	Application Programming Interface	Giao diện lập trình ứng dụng
GIL	Global Interpreter Lock	Cơ chế khóa toàn cục trong Python
OS	Operating System	Hệ điều hành
AEB	Autonomous Emergency Braking	Hệ thống phanh khẩn cấp tự động
CV	Computer Vision	Thị giác máy tính
RGB	Red – Green – Blue	Không gian màu RGB
HSV	Hue – Saturation – Value	Không gian màu HSV
HOG	Histogram of Oriented Gradients	Đặc trưng hướng gradient
IoU	Intersection over Union	Chỉ số đánh giá độ chồng khung
NMS	Non-Maximum Suppression	Loại bỏ khung trùng
Canny	Canny Edge Detector	Thuật toán phát hiện cạnh
Hough	Hough Transform	Thuật toán phát hiện đường thẳng
DNN	Deep Neural Network	Mạng nơ-ron sâu
ARM	Advanced RISC Machine	Kiến trúc vi xử lý
SoC	System on Chip	Hệ thống tích hợp trên chip

MỤC LỤC

TÓM TẮT 1
LỜI NÓI ĐẦU 2
LỜI CAM KẾT 3
DANH SÁCH HÌNH ẢNH VÀ BẢNG BIỂU 4
1. Danh sách hình 4
2. Danh sách bảng 5
DANH MỤC TỪ VIẾT TẮT 6
MỤC LỤC 7
CHƯƠNG 1: GIỚI THIỆU 9
1.1. Tính Cấp Thiết của Đề Tài 9
1.2. Trọng Tâm và Thách Thức 9
1.3. Phương Pháp Nghiên Cứu 10
1.4. Mục tiêu của đề tài 10
1.5. Phạm vi và đối tượng nghiên cứu: 10
1.6. Cấu trúc đồ án 10
Chương 2: CƠ SỞ LÝ THUYẾT 11
2.1. Tổng quan về hệ thống điện tử hỗ trợ lái xe nâng cao (ADAS) 11
2.1.1. Khái niệm 11
2.1.2. Hệ thống cảnh báo lệch làn (LDW) 12
2.1.3. Hệ thống nhận diện vật thể (Object Detection Systems - ODS) 12
2.1.4. Hệ thống phanh khẩn cấp tự động (AEB) 13
2.2. Lý thuyết về xử lý ảnh và thị giác máy tính 14
2.2.1. Các phép biến đổi ảnh cơ bản 14
2.2.2. Mạng nơ-ron Tích chập và mô hình YOLO cho bài toán Object Detection ... 18
2.3. Kết luận chương 19
Chương 3: THIẾT KẾ MÔ HÌNH HỆ THỐNG 20
3.1. Thiết kế hệ thống 20
3.1.1. Thiết kế hệ thống nhận diện vật thể 20
3.1.2. Thiết kế module nhận diện làn đường 23
3.1.3. Thiết kế mô hình nhận diện trạng thái mắt của người lái xe 27
3.2. Phân chia tiến trình xử lý trong hệ thống nhận diện đa mô hình 30

3.2.1. Phân chia tiến trình là gì 30
3.2.2. Lý do sử dụng process 30
3.2.3. Phân bổ ưu tiên và deadline cho các tác vụ 32
3.2.4. Kiến trúc thiết kế tổng thể 33
3.2.5. Đánh giá tính phù hợp với hệ thống nhúng 34
3.3. Kết luận chương 34
Chương 4: KẾT QUẢ MÔ PHỎNG VÀ ĐÁNH GIÁ 35
4.1. Đánh giá hiệu năng từng phần 35
4.1.1. Môi trường thực nghiệm và cấu hình phần cứng 35
4.1.2. Mô hình nhận diện vật thể 35
4.1.3. Mô hình nhận diện làn đường 38
4.1.4. Mô hình nhận diện trạng thái mắt 40
4.2. Đánh giá hiệu năng tổng thể 42
4.3. Kết luận chương 44
Chương 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 46
5.1. Kết luận 46
5.2. Hạn chế 46
5.3. Hướng phát triển 47
5.4. Đề xuất phần cứng 47
5.4.1. Bộ xử lý và điều khiển trung tâm 47
5.4.2. Cảm biến tích hợp 48
TÀI LIỆU THAM KHẢO 1
PHỤ LỤC 1
1. Object Detection 1
2. Object Detection (inference-only) 2
3. Eyes Detection 4
4. Eyes Detection(inference-only) 6
5. Lane Detection 7
6. Lane Detection(inference-only) 11
7. Main 13

CHƯƠNG 1: GIỚI THIỆU

1.1. Tính Cấp Thiết của Đề Tài

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của trí tuệ nhân tạo và thị giác máy tính, các hệ thống hỗ trợ lái xe thông minh (ADAS) đã trở thành một trong những công nghệ trọng tâm của ngành công nghiệp ô tô hiện đại. Theo thống kê của WHO, mỗi năm trên thế giới có hàng triệu người thương vong do tai nạn giao thông, trong đó phần lớn bắt nguồn từ lỗi của con người như mất tập trung…

Các hệ thống hỗ trợ lái xe dựa trên nhận dạng hình ảnh có khả năng phát hiện làn đường, nhận diện chướng ngại vật, phương tiện khác và giám sát trạng thái người lái theo thời gian thực. Nhờ đó, hệ thống có thể đưa ra cảnh báo hoặc hỗ trợ xử lý nhằm giảm thiểu rủi ro trong quá trình điều khiển phương tiện. Đây là nền tảng quan trọng để tiến tới các cấp độ tự hành cao hơn (Level 3–5).

Tại Việt Nam, tai nạn giao thông vẫn là vấn đề nan giải, đặc biệt tại các tuyến đường quốc lộ và khu vực đông dân cư. Tuy nhiên, các công nghệ ADAS hiện đại vẫn chưa được phổ cập rộng rãi do chi phí cao, yêu cầu tính toán lớn và phụ thuộc nhiều vào nền tảng AI. Do đó, việc nghiên cứu và mô phỏng một hệ thống hỗ trợ lái xe dựa trên nhận dạng hình ảnh thời gian thực không chỉ giúp sinh viên tiếp cận các công nghệ hiện đại như machine learning, deep learning và xử lý ảnh, mà còn mang giá trị thực tiễn trong việc phát triển các giải pháp an toàn giao thông phù hợp với điều kiện trong nước.

Vì vậy, đề tài **"Thiết kế hệ thống hỗ trợ tự lái dựa trên nhận dạng hình ảnh thời gian thực"** mang tính cấp thiết cả về mặt khoa học lẫn ứng dụng, góp phần thúc đẩy nghiên cứu và triển khai công nghệ AI trong lĩnh vực giao thông thông minh tại Việt Nam.

1.2. Trọng Tâm và Thách Thức

Bài toán trọng tâm của đề tài này là thiết kế một hệ thống hỗ trợ lái thông minh có khả năng: Nhận diện làn đường chính xác trong các điều kiện đường xá khác nhau. Phát hiện vật cản với tốc độ cao. Cuối cùng là một hệ thống giám sát trạng thái của người lái xe. Từ đó, ra cảnh báo và quyết định điều khiển an toàn.

Các thách thức chính bao gồm: Các thuật toán xử lý ảnh và AI đòi hỏi độ phức tạp tính toán lớn. Các tình huống khẩn cấp trên đường đòi hỏi thời gian phản ứng tính bằng mili giây. Hệ thống cần hoạt động hiệu quả trên nền tảng phần cứng của xe, nơi công suất tiêu thụ là một yếu tố quan trọng. Hệ thống cũng phải đảm bảo thời gian phản ứng là cố định và có thể dự đoán được trong mọi tình huống.

1.3. Phương Pháp Nghiên Cứu

Phương pháp nghiên cứu chính bao gồm: mô hình hóa hệ thống, mô phỏng thuật toán trên phần mềm Python. Đánh giá hiệu năng của hệ thống thông qua các chỉ số như FPS, độ trễ, độ chính xác và chi phí.

1.4. Mục tiêu của đề tài

Đề tài hướng tới việc xây dựng một hệ thống hỗ trợ lái xe thông minh có khả năng phân tích và xử lý hình ảnh thời gian thực nhằm nhận biết làn đường, các vật thể xung quanh và trạng thái mắt của người lái.

Từ đó, hệ thống có thể đưa ra các cảnh báo sớm khi phát hiện tình huống nguy hiểm như lệch làn, có vật cản phía trước hoặc người lái có dấu hiệu buồn ngủ/mất tập trung, góp phần nâng cao an toàn giao thông và hỗ trợ người lái trong quá trình điều khiển phương tiện.

1.5. Phạm vi và đối tượng nghiên cứu:

Phạm vi nghiên cứu:

- Nhận diện làn đường: Chỉ xét các đoạn đường quốc lộ có vạch kẻ rõ ràng, điều kiện ánh sáng tốt.
- Nhận diện vật thể: Thử nghiệm trong môi trường ánh sáng đầy đủ, các đối tượng cơ bản như xe cộ và chướng ngại vật.
- Nhận diện trạng thái người lái: thực hiện trong điều kiện ánh sáng vừa đủ, sử dụng camera cận mặt và góc nhìn tốt.

Đối tượng nghiên cứu:

- Các thuật toán nhận dạng hình ảnh thời gian thực: nhận diện vật thể, nhận diện làn đường và nhận diện mắt người lái.
- Các mô hình và kỹ thuật xử lý ảnh như YOLO, OpenCV và phương pháp tính toán EAR …
- Dữ liệu video/camera dùng cho mô phỏng hệ thống.

1.6. Cấu trúc đồ án

Cấu trúc của báo cáo bao gồm các phần như sau:

Chương 1: Giới thiệu

Chương 2: Cơ sở lý thuyết

Chương 3: Thiết kế mô hình hệ thống

Chương 4: Kết quả thực nghiệm và đánh giá

Chương 5: Kết luận và hướng phát triển.

Chương 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về hệ thống điện tử hỗ trợ lái xe nâng cao (ADAS)

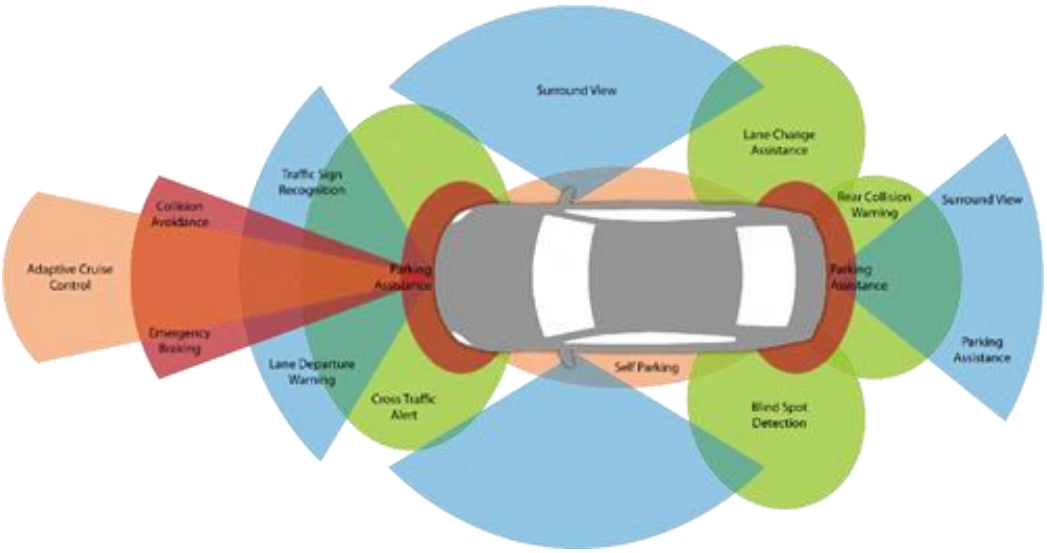
2.1.1. Khái niệm

ADAS - Advanced Driver Assistance Systems là hệ thống điện tử hỗ trợ lái xe nâng cao, giúp người lái điều khiển phương tiện an toàn hơn. Đây là các nhóm hệ thống an toàn trang bị trên xe ô tô được kết hợp giữa tính chủ động và thụ động, nhằm giảm thiểu các rủi ro do con người gây ra khi điều khiển phương tiện trên đường.

Các thành phần của hệ thống ADAS đều sử dụng những công nghệ tiên tiến nhằm hỗ trợ người lái trong quá trình vận hành, từ đó cải thiện khả năng điều khiển phương tiện của họ. ADAS sử dụng những cảm biến để nhận biết môi trường xung quanh xe, sau đó truyền thông tin đến người lái hoặc thực hiện các hành động khi cần. Điều này giúp việc điều khiển phương tiện trở nên an toàn và thuận tiện hơn, giảm áp lực cho người lái trong những tình huống xảy ra bất ngờ khi di chuyển.

Nguyên lý hoạt động của ADAS dựa trên các thông tin thu thập được từ camera đa tính năng kết hợp với công nghệ cảm biến tiên tiến. Chúng được lắp đặt bên ngoài xe, chủ yếu ở phía trước, phía sau và hai bên thân xe ô tô để ghi lại hình ảnh của nhiều đối tượng và ngôn ngữ ký hiệu giao thông như biển báo đường bộ, người đi bộ, phương tiện, đường phố và các đối tượng khác.

Các dữ liệu thu thập từ môi trường xung quanh xe sẽ được phân tích bởi hệ thống máy tính, từ đó ADAS phát hiện các nguy hiểm tiềm ẩn và đưa ra cảnh báo cũng như chủ động can thiệp trong những trường hợp người lái bị mất tập trung, góp phần nâng cao sự an toàn khi lái xe.



Hình 2.1 Ví dụ về hệ thống ADAS trong thực tiễn [1].

Hình 2.1 Mô tả một hệ thống ADAS trong thực tiễn, bằng cách sử dụng số cảm biến khổng lồ có thể giúp xe có khả năng cảm nhận và nhận biết xung quanh. Từ đó hỗ trợ tốt cho người lái.

2.1.2. Hệ thống cảnh báo lệch làn (Lane Departure Warning - LDW)

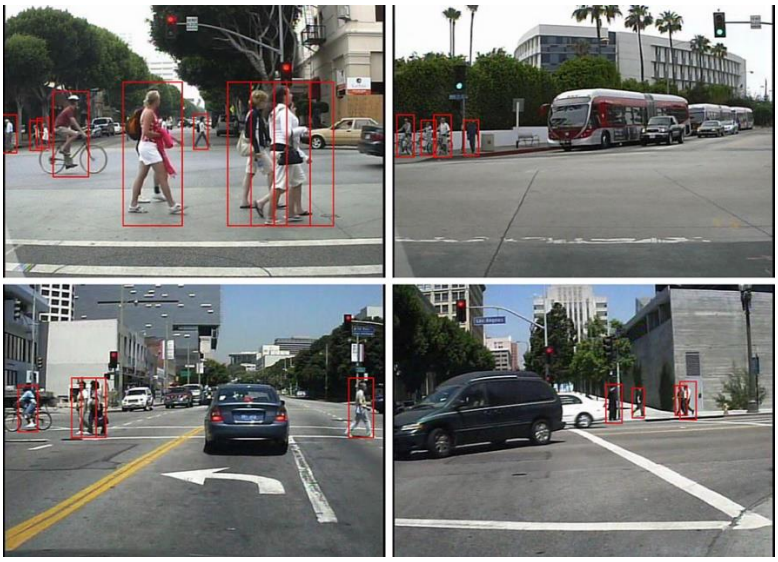
Hệ thống cảnh báo lệch làn (Hình 2.2) là một tính năng quan trọng trong các hệ thống ADAS, được thiết kế để giảm thiểu các tai nạn xảy ra khi người lái vô tình cho xe chệch khỏi làn đường hiện tại. Nguyên lý hoạt động của LDW dựa trên việc sử dụng một camera đặt ở kính chắn gió để quan sát và nhận diện các vạch kẻ đường phía trước. Hệ thống liên tục tính toán vị trí của xe so với các vạch làn đường này.

Hình 2.2 Hệ thống LDW trong thực tiễn [2].

Khi xe bắt đầu di chuyển ra khỏi làn đường mà không có tín hiệu xi-nhan (đèn báo rẽ) từ người lái, hệ thống sẽ kích hoạt cảnh báo. Cảnh báo có thể được thực hiện thông qua âm thanh, rung vô-lăng, hoặc hiển thị hình ảnh trên màn hình. Hiệu quả của LDW phụ thuộc lớn vào điều kiện đường xá (vạch kẻ đường có rõ ràng hay không) và điều kiện thời tiết (trời mưa, sương mù, nắng chói). Các thuật toán xử lý ảnh tiên tiến như Hough Transform hoặc các mô hình Deep Learning (như LaneNet) thường được sử dụng để tăng độ chính xác và độ tin cậy của hệ thống trong mọi điều kiện.

2.1.3. Hệ thống nhận diện vật thể (Object Detection Systems - ODS)

Hệ thống nhận diện người đi bộ là một hệ thống an toàn chủ động sử dụng cảm biến (thường là camera kết hợp với radar hoặc lidar) để phát hiện sự hiện diện của người đi bộ, đặc biệt là những người đang băng ngang qua hoặc di chuyển dọc theo chiều đi của xe. Khi phát hiện ra nguy cơ va chạm, hệ thống sẽ đưa ra cảnh báo kịp thời cho tài xế hoặc thậm chí tự động kích hoạt phanh trong các tình huống khẩn cấp.

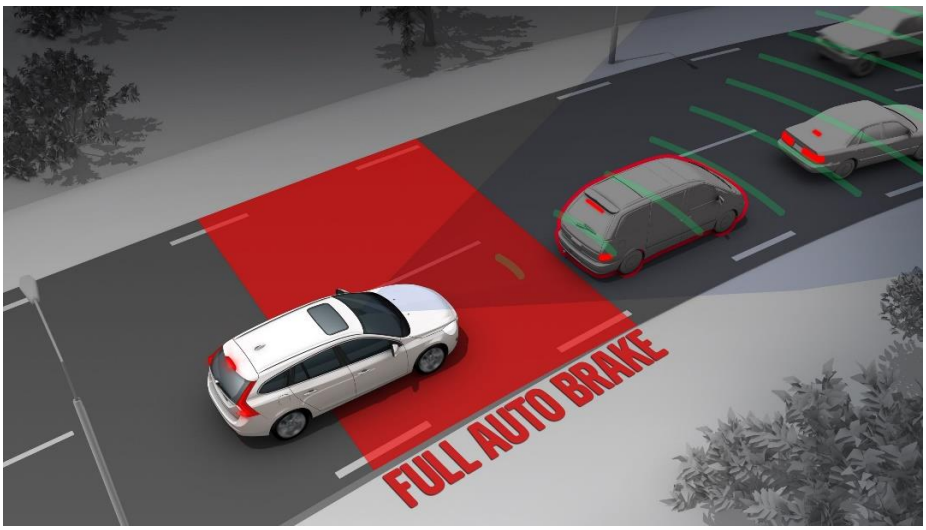


Hình 2.3 Hệ thống nhận diện vật thể [4].

Các thuật toán cho ODS chủ yếu dựa trên Thị giác máy tính (Computer Vision) và Học sâu (Deep Learning). Các mô hình phổ biến như YOLO (You Only Look Once), SSD (Single Shot MultiBox Detector), và Faster R-CNN tỏ ra rất hiệu quả trong việc nhận diện người đi bộ với tốc độ và độ chính xác cao ngay cả trong các môi trường phức tạp. Thách thức chính của PDS là khả năng hoạt động ổn định dưới các điều kiện ánh sáng yếu, thời tiết xấu, và trong các tình huống vật thể bị che khuất (occlusion).

2.1.4. Hệ thống phanh khẩn cấp tự động (Autonomous Emergency Braking - AEB)

Hệ thống phanh khẩn cấp tự động (Hình 2.4) là một hệ thống an toàn có khả năng tự động kích hoạt phanh khi phát hiện nguy cơ va chạm sắp xảy ra mà người lái không có phản ứng kịp thời. AEB hoạt động dựa trên dữ liệu từ các cảm biến như radar, lidar và camera để liên tục theo dõi khoảng cách và tốc độ tương đối giữa xe với các chướng ngại vật phía trước (xe khác, người đi bộ, vật cản).



Hình 2.4 Hệ thống phanh khẩn cấp tự động [3].

Hệ thống sử dụng các chỉ số như Thời gian đến va chạm (Time To Collision - TTC) và Khoảng cách an toàn để đánh giá mức độ nguy hiểm. Nếu TTC giảm xuống dưới một ngưỡng an toàn được xác định trước, hệ thống sẽ cảnh báo cho tài xế. Nếu tài xế vẫn không phản ứng và nguy cơ va chạm là không thể tránh khỏi, hệ thống sẽ tự động áp dụng lực phanh tối đa hoặc một phần để giảm thiểu mức độ nghiêm trọng của vụ va chạm hoặc thậm chí ngăn chặn nó hoàn toàn. AEB là nền tảng cốt lõi hướng tới các tiêu chuẩn xe tự hành hoàn toàn.

2.2. Lý thuyết về xử lý ảnh và thị giác máy tính

2.2.1. Các phép biến đổi ảnh cơ bản

Trước khi áp dụng các thuật toán phức tạp, ảnh thu được từ camera thường được xử lý qua một loạt các phép biến đổi cơ bản để cải thiện chất lượng dữ liệu đầu vào, giảm nhiễu và trích xuất các đặc trưng quan trọng. Các phép biến đổi này là nền tảng cho hầu hết các hệ thống thị giác máy tính.

2.2.1.1. Thư viện OpenCV trong Python

OpenCV (Open Source Computer Vision Library) là thư viện mã nguồn mở được sử dụng rộng rãi trong lĩnh vực thị giác máy tính và xử lý ảnh. OpenCV cung cấp hàng nghìn hàm hỗ trợ các tác vụ như đọc – ghi ảnh, xử lý ảnh, phát hiện đối tượng, trích xuất đặc trưng, theo dõi chuyển động và nhiều kĩ thuật thị giác máy tính nâng cao.

Trong Python, OpenCV được triển khai thông qua gói opencv – python, giúp lập trình viên dễ dàng xây dựng các ứng dụng xử lý ảnh nhờ cú pháp đơn giản, tốc độ xử lý cao và khả năng tích hợp tốt với các thư viện AI khác như NumPy, TensorFlow hoặc PyTorch.

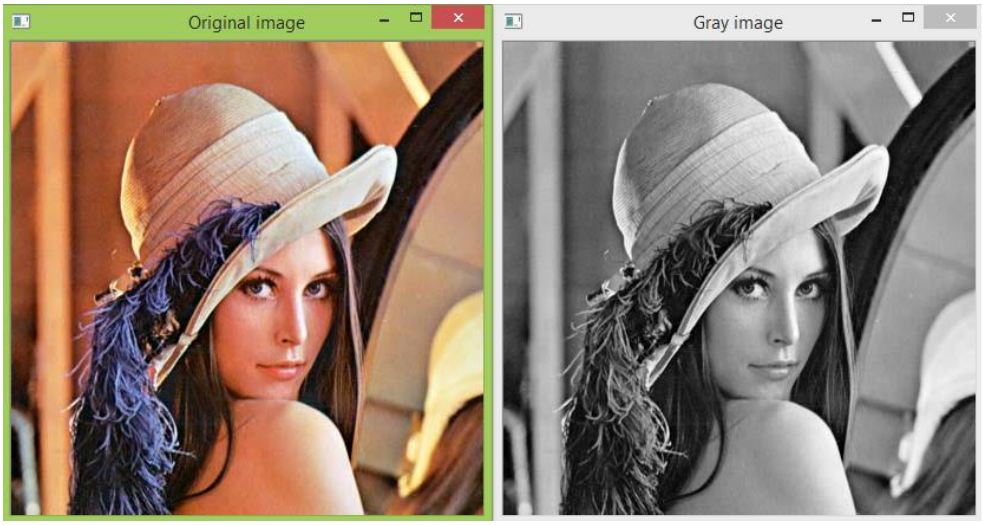
Một số chức năng chính của OpenCV trong xử lý ảnh và video bao gồm: Chuyển đổi không gian màu (RGB, HSV, Grayscale), lọc và làm mượt ảnh (Gaussian, Median), phát hiện biên (Canny), phát hiện đường thẳng (Hough Transform) …

Nhờ tính linh hoạt và hiệu năng cao, OpenCV được sử dụng làm nền tảng trong nhiều ứng dụng như giám sát giao thông, phân tích hành vi, robot tự hành và đặc biệt là các hệ thống hỗ trợ tự lái (ADAS). Trong đề tài này, OpenCV đóng vai trò quan trọng trong việc xử lý dữ liệu hình ảnh, trích rút đặc trưng và hỗ trợ các mô-đun nhận diện làn đường, vật thể và trạng thái người lái.

2.2.1.2. Chuyển đổi không gian màu

Chuyển đổi không gian màu (Color Space Conversion): Ảnh màu từ camera thường ở không gian RGB (Red-Green-Blue). Tuy nhiên, nhiều thuật toán xử lý ảnh hoạt động hiệu quả hơn trên ảnh xám (grayscale) hoặc các không gian màu khác như HSV (Hue-Saturation-Value) hoặc HLS (Hue-Lightness-Saturation).

Việc chuyển đổi từ RGB sang Grayscale làm giảm đáng kể lượng dữ liệu cần xử lý. Không gian HSV đặc biệt hữu ích cho việc tách biệt thông tin màu sắc (Hue) khỏi độ sáng (Value), giúp việc nhận diện đối tượng dựa trên màu sắc trở nên ổn định hơn dưới các điều kiện ánh sáng khác nhau.



Hình 2.5 Chuyển đổi không gian màu.

Hình 2.5 mô tả phép chuyển đổi không gian màu. Bên trái là ảnh gốc (RGB), thể hiện đầy đủ màu sắc và chi tiết tự nhiên của cảnh. Bên phải là ảnh đã được chuyển đổi sang dạng grayscale, trong đó các thông tin màu sắc được loại bỏ và thay bằng các mức độ sáng tối. Ảnh grayscale giúp nhấn mạnh cấu trúc, biên cạnh và độ tương phản của đối tượng, đồng thời giảm nhiễu và tiết kiệm tài nguyên tính toán trong các bước xử lý ảnh tiếp theo. So sánh hai ảnh cho thấy quá trình chuyển đổi grayscale giữ nguyên hình dạng tổng thể nhưng đơn giản hóa dữ liệu, phù hợp cho các thuật toán phát hiện biên, phân đoạn hoặc nhận dạng.

2.2.1.3. Làm mờ ảnh

Làm mờ ảnh (Image Blurring/Gaussian Blur): Phép toán này sử dụng một bộ lọc (kernel) để làm mờ ảnh, giúp loại bỏ nhiễu và các chi tiết không cần thiết có tần số cao. Bộ lọc Gaussian là phổ biến nhất, nó tính trung bình có trọng số các điểm ảnh lân cận, trong đó điểm ảnh ở trung tâm có trọng số cao nhất. Việc làm mờ ảnh là bước tiền quan trọng trước các phép toán như phát hiện biên để giảm thiểu khả năng phát hiện biên giả do nhiễu.

Hình 2.6 mô tả hình ảnh sau khi được sử dụng phép làm mờ ảnh. Ảnh bên trái là ảnh gốc với đầy đủ chi tiết và độ sắc nét ban đầu. Ảnh bên phải là ảnh đã được áp dụng phép làm mờ (blurring), trong đó các chi tiết nhỏ và nhiễu hình ảnh được làm giảm, tạo cảm giác mềm hơn và ít sắc cạnh hơn so với ảnh gốc. Phép làm mờ giúp loại bỏ nhiễu, giảm chi tiết không cần thiết và làm nổi bật các vùng quan trọng hơn trong quá trình xử lý ảnh. So sánh hai ảnh cho thấy ảnh làm mờ giữ lại cấu trúc tổng thể của cảnh nhưng

giảm độ sắc nét, phù hợp cho các bước như phát hiện biên, phân đoạn và loại bỏ nhiễu trước khi nhận dạng.

Hình 2.6 Phép làm mờ ảnh.

2.2.1.4. Phát hiện biên cạnh

Phát hiện biên cạnh (Edge Detection - Canny Edge Detector): Phát hiện biên là bước then chốt để tìm ra ranh giới của các đối tượng trong ảnh. Thuật toán Canny là một trong những thuật toán hiệu quả nhất cho mục đích này, bao gồm các bước: Làm mờ ảnh bằng Gaussian Blur, Tính gradient của ảnh (thường sử dụng toán tử Sobel) để tìm những nơi có sự thay đổi đột ngột về cường độ điểm ảnh (cường độ gradient lớn). Áp dụng Non-Maximum Suppression: "làm mỏng" các đường biên, chỉ giữ lại những điểm có giá trị gradient cực đại. Áp dụng Ngưỡng kép (Hysteresis Thresholding): Sử dụng hai ngưỡng (cao và thấp) để phân loại điểm ảnh thành biên mạnh, biên yếu và không phải biên. Chỉ những đường biên được kết nối với biên mạnh mới được giữ lại, giúp loại bỏ nhiễu và tạo ra các đường biên liên tục. Kết quả của phép pháp hiện biên cạnh được mô tả như hình 2.7.

Hình 2.7 Phát hiện cạnh.

2.2.1.5. Phép biến đổi góc nhìn (Perspective Transform)

Phép biến đổi góc nhìn (Perspective Transform), còn được gọi là phép chiếu ngược, là một kỹ thuật hình học quan trọng trong bài toán nhận diện làn đường. Camera trên xe thường được lắp ở một góc nghiêng để quan sát phần đường phía trước, dẫn đến hình ảnh thu được có chứa phối cảnh (các đường song song trong thực tế, như vạch kẻ đường, sẽ giao nhau tại một điểm trong ảnh). Điều này gây khó khăn cho việc tính toán khoảng cách và độ cong thực tế của làn đường.

Hình 2.8 Phép biến đổi góc nhìn.

Hình 2.8 mô tả trực quan về phép biến đổi góc nhìn. Phép biến đổi này ánh xạ ảnh từ "góc nhìn của camera" sang "góc nhìn từ trên cao xuống (Bird's-Eye View)". Bằng cách xác định 4 điểm trên ảnh gốc (tạo thành một hình thang) và ánh xạ chúng đến 4 điểm trên ảnh đích (tạo thành một hình chữ nhật), ta có thể tính được ma trận biến đổi phối cảnh.

Sau khi áp dụng ma trận này, các vạch kẻ đường gần như trở thành các đường thẳng song song, cho phép: Ước lượng chính xác hơn độ cong của làn đường. Tính toán dễ dàng độ lệch tâm (offset) của xe so với tâm làn đường. Đơn giản hóa đáng kể các bước xử lý ảnh tiếp theo như phát hiện đường thẳng bằng Hough Transform.

2.2.1.6. Kĩ thuật cửa sổ trượt (Sliding Window)

Sliding window là một kỹ thuật xử lý ảnh trong đó một cửa sổ có kích thước xác định được di chuyển tuần tự trên ảnh để tìm kiếm hoặc theo dõi các đặc trưng quan tâm. Tại mỗi vị trí của cửa sổ, hệ thống sẽ phân tích thông tin ảnh bên trong vùng đó nhằm phát hiện hoặc ước lượng vị trí của đối tượng mục tiêu.

Trong bài toán nhận diện làn đường, sliding window thường được sử dụng sau bước tiền xử lý ảnh và xác định histogram để lần theo vị trí các vạch kẻ làn. Thuật toán bắt đầu từ vị trí có mật độ điểm ảnh cao ở phần dưới ảnh, sau đó các cửa sổ được dịch chuyển dần lên phía trên theo trục dọc của ảnh, cho phép theo dõi biên làn đường một cách liên tục.

Ưu điểm của phương pháp sliding window là dễ triển khai, độ phức tạp tính toán thấp và phù hợp với các hệ thống thời gian thực chi phí thấp. Tuy nhiên, phương pháp này có thể bị ảnh hưởng bởi điều kiện ánh sáng kém, vạch kẻ đường mờ hoặc bị che khuất, dẫn đến sai lệch trong quá trình theo dõi làn đường.

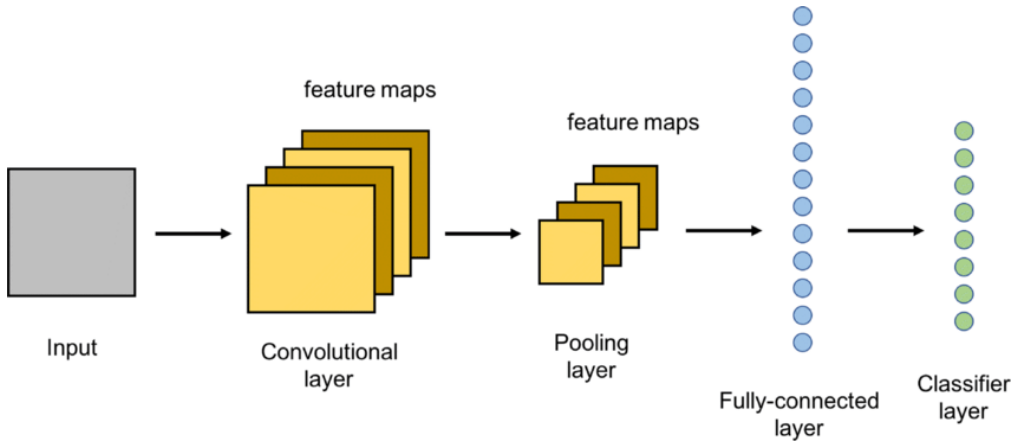
Trong đề tài này, sliding window được sử dụng như một thành phần trong thuật toán nhận diện làn đường nhằm đảm bảo tốc độ xử lý cao và khả năng hoạt động ổn định trên nền tảng CPU-only.

2.2.2. Mạng nơ-ron Tích chập và mô hình YOLO cho bài toán Object Detection

Mạng nơ-ron Tích chập (Convolutional Neural Network - CNN) là kiến trúc mạng neural tiêu chuẩn và cực kỳ hiệu quả cho các bài toán xử lý dữ liệu có dạng lưới, như hình ảnh.

Khác với mạng nơ-ron kết nối đầy đủ (Fully Connected), CNN sử dụng các phép toán tích chập (convolution) với các bộ lọc (kernel) di chuyển khắp ảnh để trích xuất tự động các đặc trưng từ cấp thấp (cạnh, góc) đến cấp cao (bộ phận khuôn mặt, bánh xe, toàn bộ đối tượng). Hình ảnh về mạng CNN được mô tả ở hình 2.9. Các thành phần chính của CNN bao gồm:

- Lớp tích chập (Convolutional Layer): Trích xuất đặc trưng.
- Lớp gộp (Pooling Layer): Giảm kích thước không gian, giảm tính toán và kiểm soát overfitting.
- Lớp kết nối đầy đủ (Fully Connected Layer): Kết hợp các đặc trưng để đưa ra dự đoán cuối cùng.



Hình 2.9 Cấu trúc CNN cơ bản.

YOLO (You Only Look Once) là một trong những họ mạng CNN hiện đại và phổ biến nhất cho bài toán Object Detection (phát hiện vật thể) theo thời gian thực. Khác với các phương pháp cũ (như R-CNN) vốn tiến hành "phát hiện" trên nhiều vùng của ảnh, YOLO áp dụng một cách tiếp cận hoàn toàn khác biệt.

Nguyên lý hoạt động: YOLO chia nhỏ ảnh đầu vào thành một lưới các ô (grid cells). Mỗi ô này chịu trách nhiệm dự đoán các khung giới hạn và độ tin cậy cho các vật thể mà tâm của vật thể nằm trong ô đó. Đồng thời, mỗi ô cũng dự đoán xác suất lớp cho vật thể.

Hình 2.10 Nguyên lý hoạt động của YOLO.

YOLO có tiềm năng đạt FPS cao khi được tối ưu và triển khai trên phần cứng phù hợp, đáp ứng yêu cầu xử lý thời gian thực trên các thiết bị nhúng và FPGA. Các phiên bản mới (YOLOv4, v5, v7, v8) đạt được sự cân bằng tuyệt vời giữa tốc độ và độ chính xác (mAP - mean Average Precision). Vì xử lý toàn bộ ảnh một lần, YOLO ít xảy ra lỗi nhận diện sai do xử lý toàn cục ảnh so với các phương pháp dựa trên vùng.

2.3. Kết luận chương

Chương này đã trình bày các kiến thức lý thuyết nền tảng quan trọng để xây dựng hệ thống nhận diện vật thể, nhận diện trạng thái mắt người lái và phát hiện làn đường trong giao thông. Các lý thuyết về thị giác máy tính, học sâu, mô hình YOLO. Cũng như các phương pháp xử lý ảnh như chuyển đổi màu, lọc nhiễu, chuyển đổi góc nhìn, cửa sổ trượt và phát hiện biên cạnh đã được giới thiệu chi tiết.

Thông qua việc phân tích và tổng hợp các kiến thức này, có thể thấy rằng các mô hình học sâu hiện nay cung cấp khả năng nhận diện nhanh và chính xác, đồng thời các kỹ thuật xử lý hình ảnh cơ bản đóng vai trò quan trọng trong việc cải thiện chất lượng dữ liệu đầu vào và độ tin cậy của hệ thống.

Chương lý thuyết đã cung cấp nền tảng cần thiết để triển khai các mô hình và thuật toán trong chương thiết kế hệ thống. Việc nắm vững các khái niệm, thuật toán và các tham số quan trọng sẽ giúp quá trình thiết kế, tối ưu hóa và đánh giá hệ thống trở nên hiệu quả hơn, đảm bảo các mục tiêu về an toàn và tốc độ xử lý trong ứng dụng thực tế.

Chương 3: THIẾT KẾ MÔ HÌNH HỆ THỐNG

3.1. Thiết kế hệ thống

3.1.1. Thiết kế hệ thống nhận diện vật thể

3.1.1.1. Giới thiệu

Hệ thống nhận diện vật thể là một thành phần quan trọng trong module hỗ trợ tự lái. Việc lựa chọn mô hình và tham số cần được cân nhắc kỹ vì ảnh hưởng trực tiếp đến an toàn của người lái và các phương tiện xung quanh.

Bài toán phát hiện vật thể nhằm xác định vị trí và loại đối tượng xuất hiện trong khung hình video hoặc ảnh đầu vào. Trong nghiên cứu này, mô hình được thiết kế dựa trên mạng, một trong những kiến trúc mạng nơ-ron tích chập phổ biến cho bài toán phát hiện đối tượng thời gian thực.

Mô hình được suy luận thông qua thư viện OpenCV DNN (Deep Neural Network), giúp thực hiện suy luận trên CPU mà không cần sử dụng framework phức tạp như TensorFlow hay PyTorch.

3.1.1.2. Mô hình toán học.

Giả sử đầu vào có kích thước WxH, các giá trị được YOLO dự đoán là:

$$(x_c, y_c, w, h)$$

Với:

- x_c(*x center*): toạ độ theo trục x của tâm bounding box (hộp giới hạn), tính từ mép trái của ảnh.
- y_c(*y center*): toạ độ theo trục y của tâm bounding box, tính từ mép trên của ảnh.
- w: chiều rộng của bounding box.
- h: chiều cao của bounding box.

Các giá trị trên được được chuẩn hoá trong khoảng từ 0 đến 1, tương ứng với tỉ lệ so với chiều rộng và chiều cao của ảnh đầu vào. Việc chuẩn hoá này giúp mô hình hoạt động ổn định với các kích thước ảnh khác nhau và thuận tiện cho quá trình huấn luyện cũng như suy luận.

Thì toạ độ góc trái trên và kích thước hộp giới hạn được tính lại theo bộ công thức (3.1):

$$x = x_c.W - \frac{w.W}{2}, \qquad y = y_c.H - \frac{h.H}{2}$$
$$w' = w.W, \qquad h' = h.H \tag{3.1}$$

Vật thể được giữ lại nếu $confidence < conf_{threshold}$

Để loại bỏ các hộp chồng lặp, ta sử dụng NMS với ngưỡng $nms_{threshold}$. Với hai hộp B_i và B_j, nếu tỉ lệ giao hợp (IoU) lớn hơn ngưỡng, hộp có confidence nhỏ hơn sẽ bị loại bỏ:

$$IoU(B_i, B_j) = \frac{area(B_i \cap B_j)}{area(B_i \cup B_j)} \tag{3.2}$$

Nếu $IoU(B_i, B_j) > nms_{threshold}$. Kết quả loại bỏ hộp có confidence thấp hơn.

Sau đó, tiến hành tính toán fps của mô hình:

$$fps = \frac{1}{t_{now} - t_{start}} \tag{3.3}$$

3.1.1.3. Lựa chọn mô hình YOLO

Giả sử trong điều kiện ô tô di chuyển với vận tốc 60km/h (vận tốc tối đa trong khu dân cư), khi có vật thể (con người, phương tiện, …). Tuy nhiên, người điều khiển phương tiện không phản ứng kịp để xử lý kịp thời. Khi đó, hệ thống phanh khẩn cấp được sử dụng để giảm thiểu rủi ro không mong muốn nhất.

Theo khảo sát, với ô tô có vận tốc khi nhỏ hơn 20km/h thì ảnh hưởng của tai nạn giao thông đến tính mạng và tài sản là đủ nhỏ (<5%). Vì vậy, ta phải xác định khoảng cách và thời gian tối đa để hệ thống phải xử lý và đưa ra quyết định. Xét xe di chuyển trên đoạn đường thẳng, không dốc và có độ bám tốt.

Trong quá trình vận hành xe, việc giảm tốc từ 60 km/h xuống 20 km/h có thể được thực hiện bằng hai phương pháp chính: phanh chủ động và nhả ga để xe giảm tốc tự nhiên. Hai phương pháp này tạo ra các mức gia tốc chậm dần khác nhau, do đó dẫn đến sự khác biệt rõ rệt về quãng đường và thời gian giảm tốc.

Công thức 3.4 tính toán khoảng cách tối đa mà xe phải thực hiện dự đoán và đưa quyết định đối với gia tốc đều:

$$v_0^2 - v_1^2 = 2as \tag{3.4}$$

Vậy khoảng cách tối đa mà xe phải xử lý là

$$s = \frac{v_0^2 - v_1^2}{2a} \tag{3.5}$$

Bảng 3.1 Mối quan hệ giữa gia tốc, quãng đường giảm tốc và thời gian

Gia tốc a (m/s²)	Quãng đường giảm tốc s (m)	Thời gian t (s)
-4	30.86	2.78
-6	20.58	1.85
-8	15.43	1.39
-10	12.35	1.11

Theo khảo sát của Applus IDADA được công bố trên The Brake Report [11], các dữ liệu thực nghiệm cho thấy mức giảm tốc phổ biến của một người lái xe có kỹ năng tốt vào khoảng –7 m/s². Với mức giảm tốc này, khi xe đang di chuyển ở vận tốc 60 km/h và tốc độ tại thời điểm va chạm giảm xuống 20 km/h, quãng đường tối đa cần thiết để xe giảm tốc nằm trong khoảng 18 m.

Tuy nhiên, theo quy định hiện hành tại Việt Nam, khoảng cách an toàn tối thiểu giữa hai ô tô khi di chuyển ở vận tốc 60 km/h phải đạt 35 m. Điều này cho thấy trên thực tế, người lái thường duy trì khoảng cách ngắn hơn quy định do tâm lý chủ quan, khiến khoảng cách đó có thể đủ an toàn cho phương tiện của họ nhưng lại không đảm bảo an toàn cho các phương tiện xung quanh hoặc người đi bộ. Do đó, nguy cơ xảy ra va chạm vẫn hiện hữu.

Từ thực tế này, mục tiêu đặt ra cho mô hình AI trong đề tài là có thể nhận diện chính xác các đối tượng trong phạm vi 10 m phía trước, bảo đảm đủ thời gian phản ứng để hỗ trợ người lái và nâng cao mức độ an toàn khi tham gia giao thông.

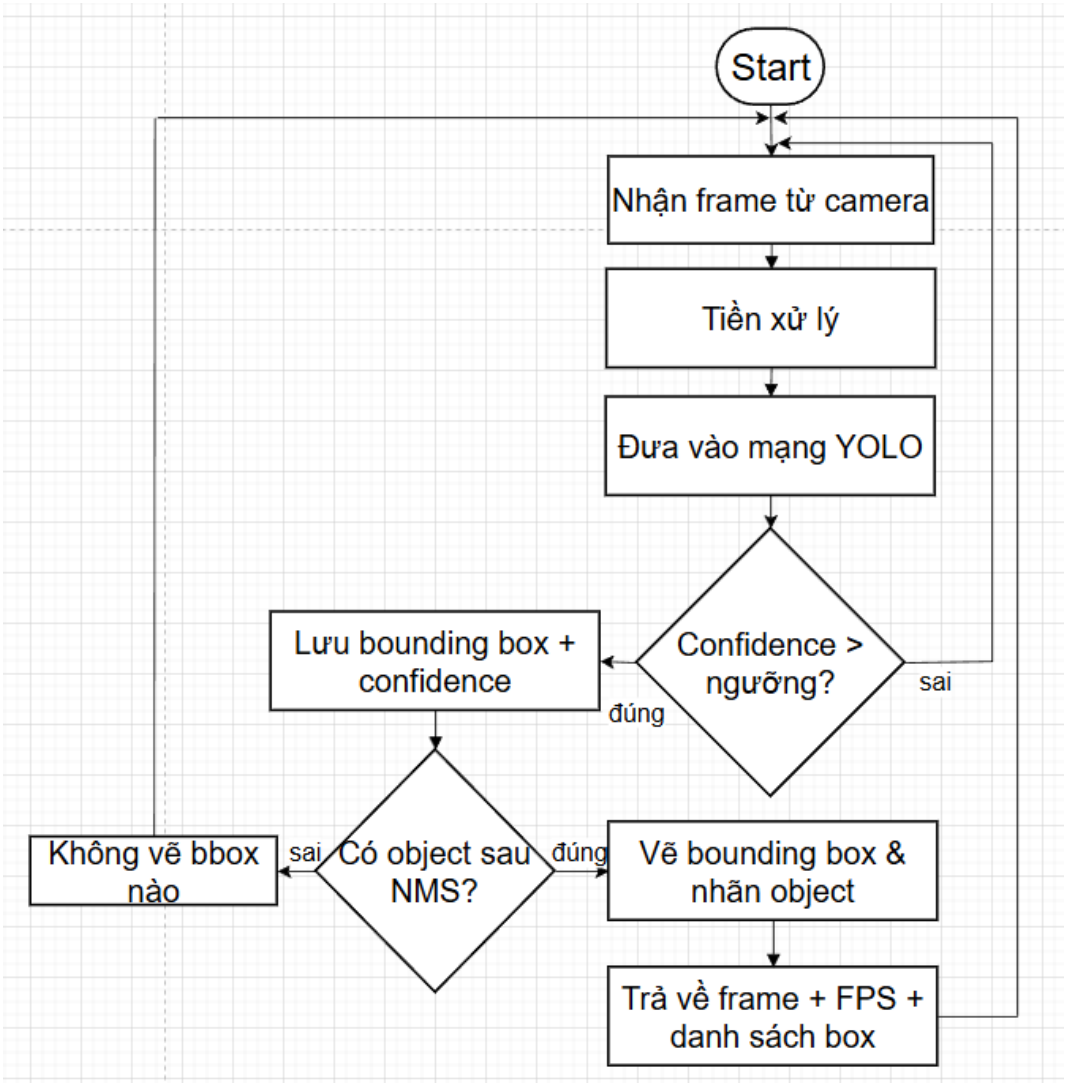
Trong quá trình thực nghiệm, mô hình YOLOv3 cho độ chính xác cao nhưng tốc độ xử lý chỉ đạt 2-3 FPS, không phù hợp với yêu cầu xử lý thời gian thực của hệ thống hỗ trợ lái xe. Khi thay thế bằng mô hình YOLOv3-tiny, tốc độ xử lý tăng lên 15–20 FPS, đồng thời vẫn duy trì độ chính xác ổn định (chi tiết được trình bày ở phần kết quả thực nghiệm).

Vì vậy, với yêu cầu về tốc độ, hiệu quả kinh tế, và khả năng nhận diện chính xác trong phạm vi mục tiêu, mô hình YOLOv3-tiny được lựa chọn là giải pháp phù hợp và đáp ứng đầy đủ các tiêu chí của hệ thống.

3.1.1.4. Lưu đồ thuật toán của mô hình

Thuật toán nhận diện vật thể bắt đầu bằng việc đọc một khung hình và chuẩn hóa kích thước ảnh để phù hợp với mô hình YOLO. Trên ảnh đầu vào, hệ thống vẽ các điểm góc tham chiếu nhằm xác định vùng quan tâm. Ảnh sau đó được chuyển thành blob và đưa vào mạng YOLOv3-tiny để thực hiện suy luận.

Các giá trị dự đoán được trích xuất và sàng lọc dựa trên ngưỡng độ tin cậy. Những khung bao đạt yêu cầu được lưu lại và tiếp tục qua bước non-maximum suppression nhằm loại bỏ các khung bao bị trùng lặp. Nếu còn lại các đối tượng hợp lệ, hệ thống tiến hành vẽ bounding box và gán nhãn lên ảnh. Cuối cùng, tốc độ xử lý (FPS – Frame Per Second) được tính toán và trả về cùng kết quả phát hiện vật thể.



Hình 3.1 Lưu đồ thuật toán của mô hình nhận diện vật thể.

3.1.2. Thiết kế module nhận diện làn đường

3.1.2.1. Giới thiệu về mô hình

Module nhận diện làn đường cũng là một trong những module quan trọng trong kiến trúc này. Việc nhận diện làn đường giúp việc cân bằng tự động trong việc di chuyển của xe, đưa cảnh báo cho người điều khiển khi chưa có tín hiệu chuyển làn nhưng đi lệch so với làn đường rất nhiều.

Hệ thống nhận diện làn đường được xây dựng dựa trên kỹ thuật xử lý ảnh trong OpenCV nhằm phát hiện và theo dõi biên dạng làn đường từ camera gắn phía trước xe. Thuật toán sử dụng kết hợp biến đổi phối cảnh, phân ngưỡng theo không gian màu HSV, histogram và sliding Window để xác định vị trí vạch kẻ đường trong thời gian thực. Mô hình hỗ trợ việc tính toán độ lệch của xe so với tâm làn và góc đánh lái ước lượng, từ đó phục vụ các hệ thống hỗ trợ lái xe.

3.1.2.2. Mô hình toán học

Biến đổi phối cảnh được mô tả bằng ma trận 3x3:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \boldsymbol{H} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \text{ với } \boldsymbol{H} \in \boldsymbol{R}^{3x3} \quad (3.6)$$

Trong đó:

- (x, y) là điểm ở không gian gốc
- (x',y') là điểm sau khi chuyển về góc nhìn từ trên xuống (Bird-Eye View)
- ***H*** được ước lượng từ 4 điểm tương ứng (tl, tr, bl, br). Với tl = (250, 350), tr = (390, 350), bl = (80, 472), br = (560, 472).

Sau đó ảnh được chuyển sang không gian HSV và phân loại ngưỡng:

$$Mark(i,j) = \begin{cases} 1 \ \ nếu\ L \leq HSV(i,j) \leq U \\ 0 \ \ ngược\ lại \end{cases} \quad (3.7)$$

Mark(i, j) biểu diễn giá trị của điểm ảnh tại vị trí (i, j) trong ma trận nhị phân sau tiền xử lý, trong đó giá trị khác 0 được dùng để đánh dấu các điểm ảnh có khả năng thuộc biên làn đường.

Với L = [L-H, L-S, L-V] và U = [U-H, U-S, U-V]. Các tham số trên được điều chỉnh real-time bằng Trackbar.

Bảng 3.2 Các tham số trong không gian HSV

Tham số	Ý nghĩa	Giá trị khởi tạo
L-H	Hue thấp	0
L-S	Saturation thấp	0
L-V	Value thấp	113
U-H	Hue cao	255
U-S	Saturation cao	43
U-V	Value cao	255

Tuy nhiên, giới hạn trong báo cáo này. Việc tối ưu hoá tất cả tham số trong bộ chỉnh màu là khá phức tạp. Vì vậy, hệ thống sẽ chỉ tự động kiểm tra tỷ lệ màu trắng trong mask và điều chỉnh ngưỡng L-V để thích nghi với độ sáng môi trường. Cụ thể:

$$ratio_{white} = \frac{countNonZero(mask)}{mask.size} \quad (3.8)$$

Sau khi ảnh được nhị phân hoá. Giả sử ảnh lúc này là một mảng bao gồm các phần tử 0 và 1. Hay ảnh lúc này được mô tả:

$$I(x,y) \in \{0,1\} \quad (3.9)$$

Sau đó, ta tính histogram theo cột (chỉ lấy nửa dưới ảnh). Vì vạch làn đường xuất phát từ dưới và khoảng trên của ảnh thì vạch làn không còn đủ rõ để nhận diện.

$$H(x) = \sum I(x,y) \quad (3.10)$$

Lane base là vị trí hay là cột có nhiều điểm trắng nhất. Một cách đơn giản là cột nào sáng nhiều thì khả năng nó là chân của vạch kẻ.

Từ hai điểm base tìm được, thuật toán chia ảnh thành nhiều "tầng" từ dưới lên và đặt hai cửa sổ để tìm pixel của vạch. Ở mỗi tầng, thực hiện đặt cửa sổ quanh vị trí lane base. Sau đó, tìm tất cả điểm trắng ở trong cửa sổ. Nếu có điểm trắng, cửa sổ của tầng kế tiếp sẽ dịch sang vị trí trung bình của điểm này. Giúp nó bám theo đường cong của vạch kẻ. Lặp lại đến khi hết chiều cao của ảnh.

Thuật toán lần lượt xét từng tầng từ dưới lên; tại mỗi tầng, cửa sổ trượt dò tìm vị trí vạch và dịch chuyển theo điểm trung bình nhằm bám theo đường cong của làn đường.

Sau khi thu thập được một tập hợp các điểm lane từ làn trái và phải. Ta cần ước lượng hình dạng thật của hai vạch này. Vạch kẻ đường thường không thẳng hoàn toàn mà có độ cong nhất định. Vì vậy, ta sử dụng đa thức bậc hai để mô hình hoá mỗi vạch:

$$x = ay^2 + by + c \tag{3.11}$$

Trong đó:

- x: vị trí ngang của vạch kẻ có độ cao y.
- a,b,c: các hệ số được tính toán từ dữ liệu điểm thu được.

Quá trình này được thực hiện bằng hàm polyfit của NumPy, vốn dùng phương pháp bình phương tối thiểu (Least Squares) để tìm đường cong bậc 2 khớp tốt nhất với các điểm vạch.

Tính hệ số bằng polyfit:

$$\begin{aligned} leftfit &= polyfit(y, lx, 2) \\ rightfit &= polyfit(y, rx, 2) \end{aligned} \tag{3.12}$$

Kết quả trả ra là các giá trị a,b. Từ đó, ước lượng độ cong (curvature):

$$K = \frac{(1 + (2ay + b)^2)^{\frac{3}{2}}}{|2a|} \tag{3.13}$$

Sau đó tìm tâm của làn đường mô hình xác định được bằng:

$$lanecenter = \frac{leftbase + rightbase}{2} \tag{3.14}$$

Độ lệch ngang off-set được xác định:

$$offset = \frac{(320 - lanecenter)}{640px} \times 3.7 \tag{3.15}$$

Vì frame được chuẩn hoá về 640x320 nên offset là khoảng cách giữa làn trung tâm được nhận diện và tâm của frame. Ngoài ra, 3.7 là độ rộng làn đường cơ bản.

Cuối cùng, góc lái được tính toán bằng:

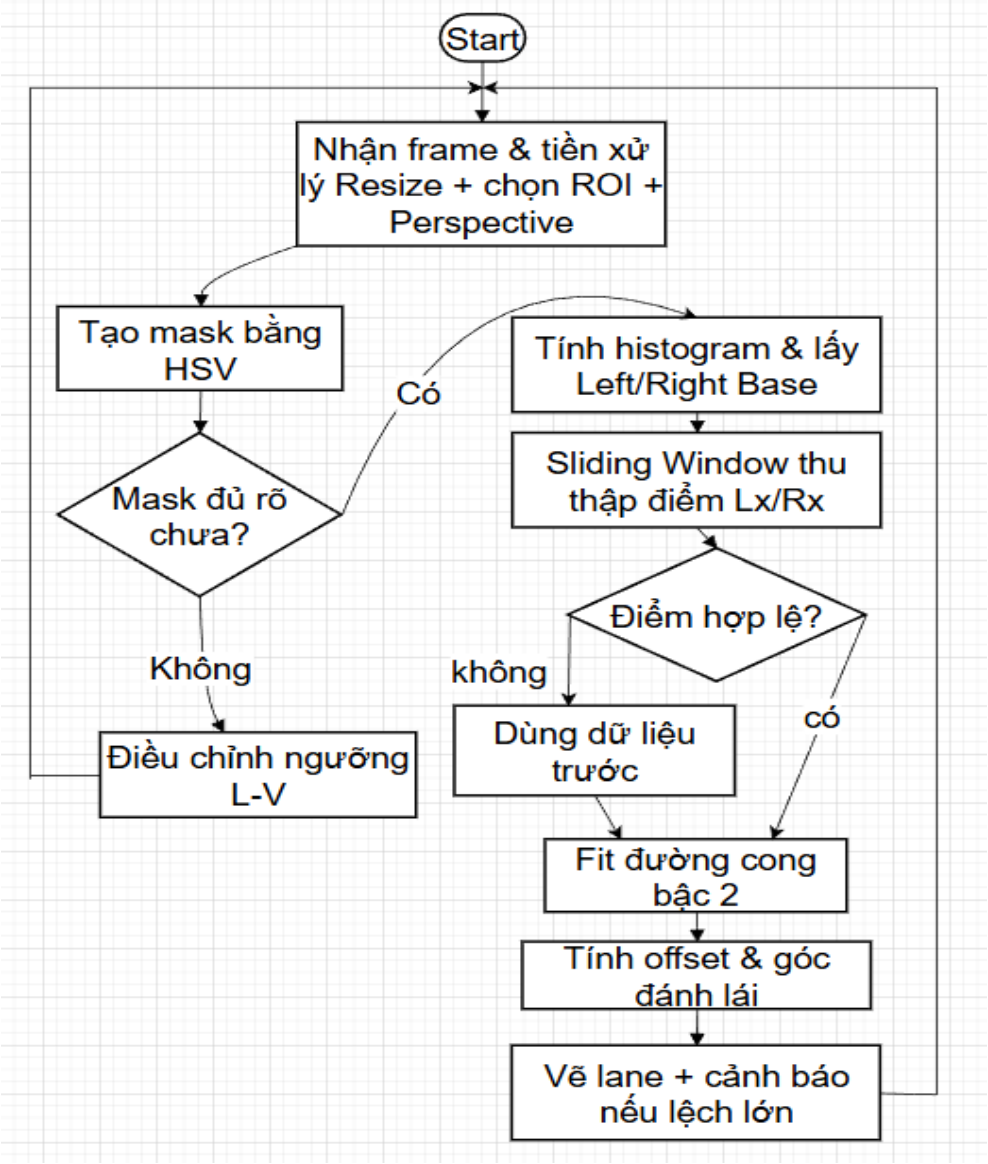
$$\theta = \frac{offset}{curvature} \tag{3.16}$$

Tính toán fps của hệ thống bằng công thức:

$$FPS = \frac{1}{t_{now} - t_{start}} \tag{3.17}$$

3.1.2.3. Lưu đồ thuật toán của mô hình

Hình 3.2 mô tả Sơ đồ thuật toán về quy trình xử lý của hệ thống phát hiện và theo dõi làn đường trong môi trường thời gian thực. Khi hệ thống khởi động, camera phía trước xe cung cấp liên tục các khung hình cho thuật toán. Khung hình này được đưa vào giai đoạn tiền xử lý nhằm chuẩn hóa dữ liệu đầu vào, bao gồm chuyển đổi không gian màu, làm mịn và tăng cường độ tương phản để làm rõ các đặc điểm của mặt đường.



Hình 3.2 Lưu đồ thuật toán của mô hình nhận diện làn đường.

Sau bước tiền xử lý, thuật toán tiến hành phát hiện biên bằng các phương pháp như Canny nhằm làm nổi bật cấu trúc đường và vạch kẻ. Để giảm nhiễu không cần thiết, khu vực ngoài phạm vi mặt đường được loại bỏ thông qua việc xác định vùng quan tâm (Region of Interest).

Tiếp theo, ảnh được biến đổi sang dạng quan sát từ trên xuống (Bird-Eye View) bằng phép biến đổi phối cảnh, giúp làm thẳng vạch kẻ và tạo điều kiện thuận lợi cho việc phân tích.

Trong không gian bird-eye, hệ thống sử dụng phương pháp cửa sổ trượt kết hợp với histogram để xác định các điểm pixel thuộc làn đường trái và phải. Các điểm này được gom nhóm và sử dụng để ước lượng đường cong của làn bằng mô hình đa thức bậc hai, nhằm tạo ra đường làn mượt và ổn định theo thời gian.

Kết quả mô hình hóa sau đó được ánh xạ ngược trở lại ảnh gốc và trực quan hóa bằng cách tô sáng vùng làn đường. Cuối cùng, hệ thống quay trở lại bước đọc khung hình mới và lặp lại toàn bộ quy trình, bảo đảm khả năng hoạt động liên tục trong thời gian thực.

3.1.3. Thiết kế mô hình nhận diện trạng thái mắt của người lái xe

3.1.3.1. Giới thiệu về mô hình

Trong hệ thống cảnh báo buồn ngủ của người lái xe, việc nhận diện trạng thái mắt mở hay nhắm là một trong những yếu tố quan trọng để phát hiện tình trạng mệt mỏi.Mô hình được xây dựng nhằm xác định trạng thái của mắt theo thời gian thực thông qua dữ liệu video thu được từ camera giám sát, sau đó đánh giá mức độ tập trung của người lái.

Để trích xuất đặc trưng khuôn mặt, hệ thống sử dụng thư viện MediaPipe FaceMesh của Google, kết hợp với thuật toán Eye Aspect Ratio (EAR) để mô hình hóa hình học của mắt và đưa ra quyết định phân loại.

3.1.3.2. Thư viện MediaPipe FaceMesh

MediaPipe FaceMesh là một mô hình học sâu (Deep Learning Model) được huấn luyện sẵn để phát hiện và định vị 468 điểm mốc (landmarks) trên khuôn mặt người trong không gian 2D hoặc 3D theo thời gian thực.

Ưu điểm nổi bật của FaceMesh bao gồm: Tốc độ xử lý cao: có thể chạy trên CPU thời gian thực mà không cần GPU. Độ chính xác tốt: dựa trên mô hình mạng nơ-ron tích chập (CNN) đã được huấn luyện trên lượng lớn dữ liệu khuôn mặt. Khả năng xác định chi tiết vùng mắt, mũi, môi, cằm, và viền khuôn mặt.

Cơ chế hoạt động của MediaPipe FaceMesh bao gồm ba giai đoạn chính: Đầu tiên, xác định vị trí khuôn mặt trong khung hình. Sau đó, xác định tọa độ (x, y, z) của

468 điểm mốc. Cuối cùng, theo dõi sự thay đổi vị trí của các điểm mốc qua từng khung hình để đảm bảo chuyển động mượt và ổn định.

Trong bài toán này, chỉ một tập con các điểm mốc quanh mắt được sử dụng để tính toán chỉ số EAR: Mắt trái: [33, 160, 158, 133, 153, 144]. Mắt phải: [362, 385, 387, 263, 373, 380]

Hình 3.3 Các điểm landmarks được xác định trên mặt.

3.1.3.3. Mô hình toán học

Để mô tả trạng thái của mắt, ta sử dụng chỉ số Eye Aspect Ratio (EAR) được định nghĩa ở công thức như sau.

$$EAR = \frac{\left||P_2 - P_6|\right| + ||P_3 - P_5||}{2\,.\,||P_1 - P_4||} \tag{3.18}$$

Trong đó:

P1,P2, …, P6 là các điểm mốc quanh mắt được xác định bởi MediaPipe FaceMesh.

||A-B|| là khoảng cách Euclidean giữa 2 điểm A và B:

$$\left||A - B|\right| = \sqrt{(x_A - x_B)^2 + (y_a - y_b)^2} \tag{3.19}$$

Khi mắt mở, khoảng cách dọc giữa mí trên và mí dưới lớn, dẫn đến giá trị EAR cao. Khi mắt nhắm, các khoảng cách giảm mạnh, khiến EAR giảm xuống dưới một ngưỡng nhất định.

Sau đó tính toán EAR cho từng mắt. Khi đó, mô hình phân loại nhị phân được thiết lập như sau:

$$state = \begin{cases} close\ (EAR_{left} < \ \theta\ và\ EAR_{rigth} < \theta) \\ open\ (AR_{left} \geq \ \theta\ và\ EAR_{rigth} \geq \theta) \end{cases} \tag{3.20}$$

Trong đó:

- θ là ngưỡng EAR xác định theo thực nghiệm (ví dụ: theta = 0.25).
- "close" biểu thị mắt nhắm, "open" biểu thị mắt mở

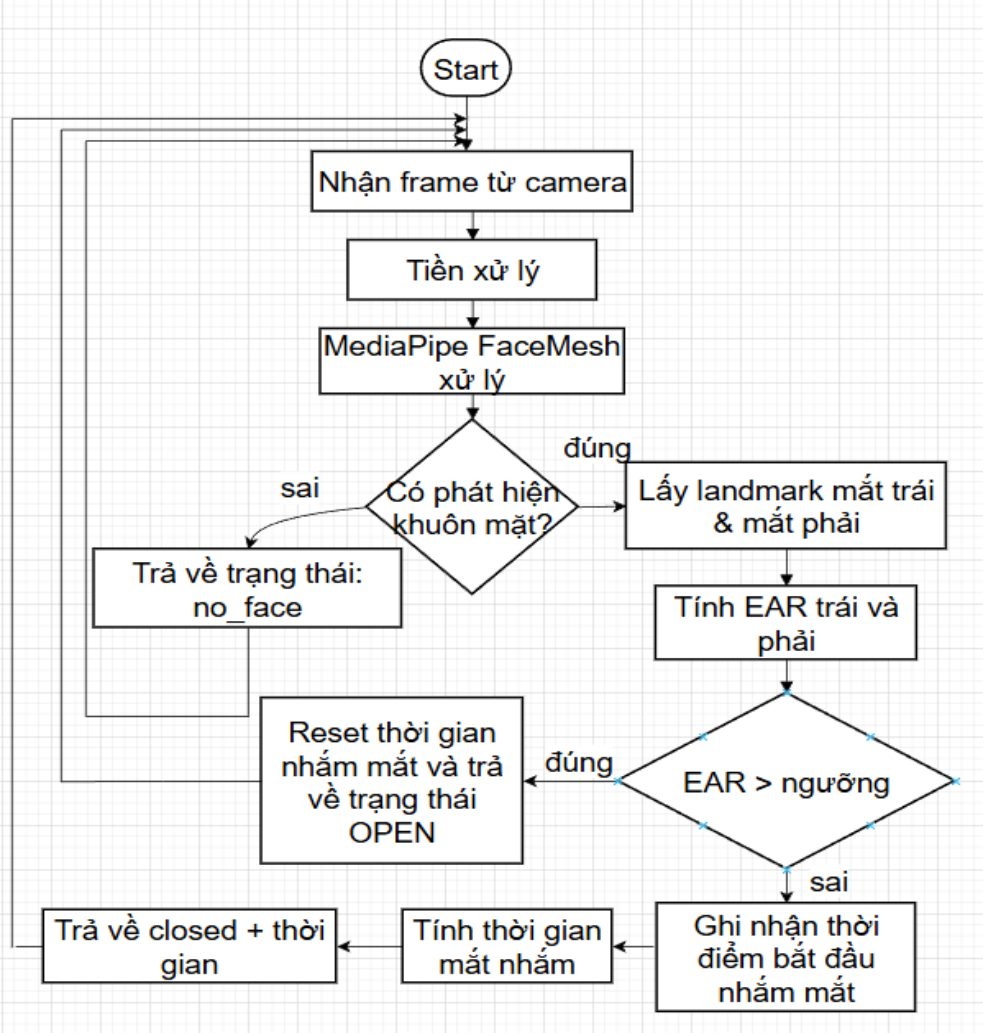
Ngoài ra việc xác định tình trạng buồn ngủ, mô hình không chỉ xét trạng thái tức thời mà còn theo dõi thời gian nhắm mắt liên tục. Nếu mắt được xác định "nhắm - close", hệ thống bắt đầu ghi nhận thời điểm t_{start}. Khi mắt tiếp tục nhắm, thời gian nhắm mắt được tính:

$$T_c = t_{curent} - t_{start} \quad (3.21)$$

Nếu giá trị T_c vượt quá một ngưỡng thời gian $t_{threshold}$ (3 giây), hệ thống kết luận người lái có dấu hiện buồn ngủ hoặc ngủ gật.

3.1.3.4. Mô hình hoạt động tổng quát

Mô hình hoạt động được mô tả bằng lưu đồ ở hình 3.4. Quá trình bắt đầu bằng việc nhận một khung hình từ camera và chuyển hình sang hệ không gian màu RGB để đưa vào mô hình FaceMesh của MediaPipe. Nếu không phát hiện được khuôn mặt, hệ thống lập tức trả về trạng thái "no_face".



Hình 3.4 Lưu đồ thuật toán của mô hình nhận diện trạng thái mắt.

Khi nhận diện được mặt, thuật toán lấy các điểm đặc trưng quanh mắt trái và mắt phải, sau đó tính chỉ số Eye Aspect Ratio (EAR) cho từng mắt. EAR được so sánh với một ngưỡng đặt trước; nếu cả hai mắt đều có EAR nhỏ hơn ngưỡng thì hệ thống xác định trạng thái "nhắm mắt". Khi mắt nhắm liên tục, thời điểm bắt đầu nhắm được lưu lại và thời gian nhắm mắt được tính theo thực thời. Nếu mắt mở trở lại, bộ đếm thời gian được đặt lại và trả về trạng thái "open".

Trường hợp mắt nhắm kéo dài quá thời gian cho phép có thể được dùng để phát hiện dấu hiệu buồn ngủ.

3.2. Phân chia tiến trình xử lý trong hệ thống nhận diện đa mô hình

3.2.1. Phân chia tiến trình là gì

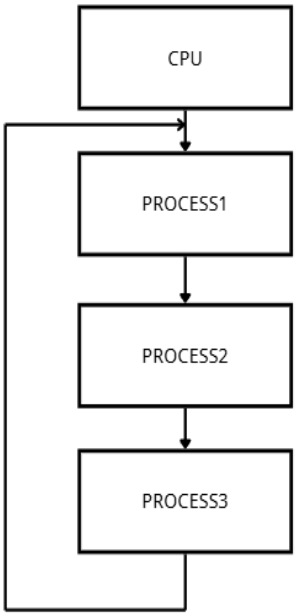
Trong hệ thống nhận diện đa mô hình đều có đặc điểm chung: đều yêu cầu xử lý ảnh theo thời gian thực và tiêu tốn tài nguyên tính toán lớn. Nếu gộp toàn bộ các tác vụ này vào một luồng xử lý duy nhất, hệ thống sẽ nhanh chóng rơi vào tình trạng nghẽn (bottleneck), khiến tốc độ suy giảm nghiêm trọng.

Để bảo đảm mỗi mô hình được vận hành ổn định, hệ thống lựa chọn cơ chế đa tiến trình (multiprocessing) nhằm tách riêng từng tác vụ nặng thành một tiến trình độc lập. Mỗi tiến trình sẽ được cấp phát vùng nhớ và tài nguyên CPU/GPU riêng, giúp giảm thiểu xung đột, đồng thời bảo đảm tốc độ xử lý thời gian thực cho từng mô-đun.

3.2.2. Lý do sử dụng process

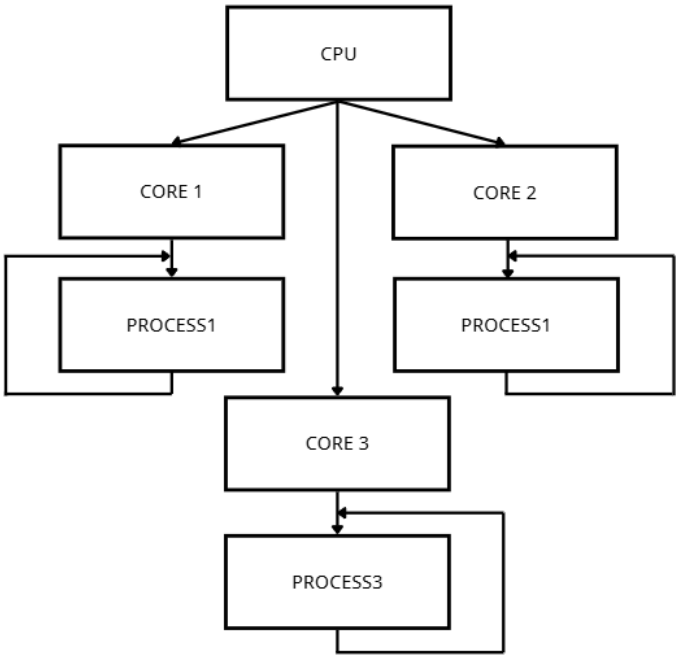
Với các tác vụ dùng Python như YOLO, xử lý bằng CNN hoặc OpenCV, việc chạy bằng đa luồng (multithreading) không làm tăng hiệu năng do bị giới hạn bởi GIL. Điều này khiến dù có nhiều luồng thì thực tế vẫn chỉ một luồng được truy cập CPU tại mỗi thời điểm. Ngược lại, multiprocessing tạo ra các tiến trình độc lập, không chịu ảnh hưởng của GIL, giúp tận dụng hiệu quả toàn bộ số nhân CPU.

Hình 3.5 minh họa cấu trúc của một chương trình không sử dụng đa tiến trình. Trong mô hình này, chương trình chính phải thực hiện tuần tự qua từng hệ thống con, khiến thời gian xử lý của mỗi mô hình phụ thuộc vào các mô hình còn lại. Điều này dẫn đến độ trễ tổng thể tăng và làm giảm tốc độ phản hồi của toàn hệ thống.



Hình 3.5 Cấu trúc phần cứng khi không sử dụng đa tiến trình.

Việc sử dụng đa tiến trình (Hình 3.6) cho phép các mô hình hoạt động độc lập, tận dụng tối đa tài nguyên xử lý đa nhân, giảm độ trễ và tăng khả năng phản hồi của hệ thống.



Hình 3.6 Cấu trúc phần cứng khi sử dụng đa tiến trình.

Nếu một mô hình gặp lỗi (treo GPU, lỗi đọc camera…), toàn bộ hệ thống sẽ không bị dừng lại. Với đa tiến trình, hệ thống có thể khởi động lại tiến trình lỗi mà không ảnh hưởng đến các mô-đun còn lại.

Ngoài ra, khi cần thêm mô-đun như cảnh báo khoảng cách, theo dõi tốc độ, phân tích hành vi… ta chỉ việc bổ sung tiến trình mới mà không phải chỉnh sửa cấu trúc chung của chương trình.

Ngoài việc sử dụng process, một cách khác để chạy các mô hình song song là dùng thread. Tuy nhiên, theo Techmaster.vn, việc so sánh giữa process và thread được trình bày cụ thể trong Bảng 3.3:

Bảng 3.3 So sánh sử dụng Process và Thread

Process	Thread
Tối ưu cho CPU/GPU-bound	Hạn chế
Tách riêng bộ nhớ cho từng tiến trình	Dùng chung địa chỉ nhớ
Tiến trình này bị treo không bị ảnh hưởng tiến trình khác	Làm treo cả hệ thống
Sử dụng phức tạp	Sử dụng đơn giản
Độ ổn định cao	Cẩn thận trong quá trình thực hiện

Như vậy, với các ưu điểm về khả năng tận dụng CPU/GPU, độ ổn định, tách riêng bộ nhớ và giảm thiểu rủi ro khi một tiến trình gặp sự cố, việc sử dụng process được đánh giá phù hợp hơn cho hệ thống nhận diện đa mô hình trong ứng dụng ADAS.

3.2.3. Phân bổ ưu tiên và deadline cho các tác vụ

Hệ thống không yêu cầu tất cả các tác vụ phải hoàn thành trong cùng một chu kỳ xử lý. Trong trường hợp CPU bị quá tải, các khung hình mới sẽ thay thế các khung hình cũ trong hàng đợi (queue size = 1). Cơ chế này giúp hệ thống luôn xử lý dữ liệu mới nhất, tránh hiện tượng tích lũy độ trễ và đảm bảo đặc tính thời gian thực mềm (soft real-time) của hệ thống.

Để duy trì tính ổn định trong điều kiện tải cao, hệ thống áp dụng các cơ chế scheduling và quản lý tài nguyên nhằm đảm bảo đặc tính thời gian thực mềm. Khi tốc độ xử lý của hệ thống thấp hơn tốc độ thu nhận khung hình từ camera, các khung hình cũ sẽ được bỏ qua và thay thế bằng các khung hình mới trong hàng đợi. Cơ chế này giúp hệ thống luôn xử lý dữ liệu cập nhật nhất, hạn chế hiện tượng tích lũy độ trễ và duy trì khả năng phản hồi kịp thời.

Bên cạnh đó, các mô-đun trong hệ thống được thiết kế theo hướng xử lý bất đồng bộ và hoạt động độc lập với nhau. Mỗi mô-đun có thể vận hành ở tần số phù hợp với đặc điểm xử lý riêng, tránh tình trạng một tác vụ chậm gây ảnh hưởng đến toàn bộ hệ thống. Việc tách biệt các mô-đun thành các tiến trình độc lập cũng góp phần nâng cao độ ổn định, cho phép các thành phần còn lại tiếp tục hoạt động bình thường ngay cả khi một mô-đun gặp lỗi hoặc bị trễ xử lý.

Nhờ các cơ chế trên, hệ thống đảm bảo khả năng phản hồi trong giới hạn thời gian cho phép và duy trì hoạt động ổn định trên nền tảng CPU-only, đáp ứng yêu cầu của một hệ thống ADAS thời gian thực mềm.

3.2.4. Kiến trúc thiết kế tổng thể

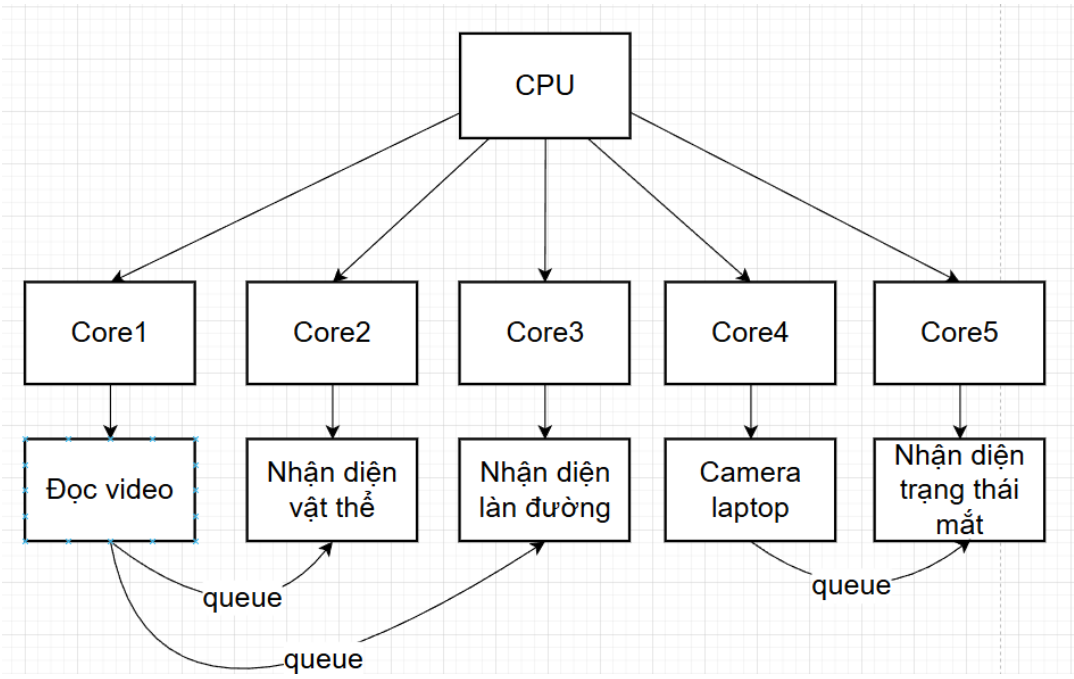
Hệ thống nhận diện đa mô hình được thiết kế theo kiến trúc đa tiến trình để tối ưu hóa hiệu năng xử lý hình ảnh từ camera trong thời gian thực. Mỗi tiến trình đảm nhận một nhiệm vụ riêng biệt, giúp giảm thiểu tắc nghẽn xử lý, tận dụng tối đa khả năng đa nhân của CPU và tăng FPS. Mỗi tiến trình đảm nhiệm một chức năng riêng, dữ liệu được trao đổi giữa các tiến trình thông qua queue, tránh xung đột bộ nhớ và tăng tính ổn định cho hệ thống.

Cấu trúc tổng thể gồm các thành phần được mô tả ở hình 3.7. Process đầu tiên đọc video từ file video (mô phỏng cam trước ô tô), nhiệm vụ nhận luồng video phục vụ cho nhận diện vật thể. Sau đó dữ liệu được gửi qua queue đến các process xử lý nhận diện.

Trong sơ đồ minh họa, mỗi tiến trình được gán tương ứng với một nhân CPU để làm rõ tính song song của hệ thống. Trên thực tế, việc phân phối tiến trình lên các nhân CPU được hệ điều hành quản lý động thông qua cơ chế scheduling.

Tiếp theo là process nhận diện làn đường và process nhận diện vật thể. Nhận dữ liệu từ process đầu và chạy mô hình nhận diện trên mỗi luồng riêng của nó.

Tương tự với process đọc camera laptop thu nhận luồng video trực tiếp từ camera gửi đến process nhận diện trạng thái mắt và chạy mô hình đó.



Hình 3.7 Kiến trúc thiết kế cho hệ thống tổng thể.

3.2.5. Đánh giá tính phù hợp với hệ thống nhúng

Mặc dù hệ thống được triển khai trên laptop trong môi trường mô phỏng, kiến trúc scheduling và phân tích thời gian thực đã được thiết kế theo hướng sẵn sàng triển khai trên nền tảng nhúng. Các đặc điểm như phân chia tác vụ độc lập, deadline rõ ràng và cơ chế xử lý quá tải là những yếu tố quan trọng trong các hệ thống ADAS thực tế.

Việc tối ưu backend suy luận và triển khai trên các nền tảng nhúng trong tương lai (như Jetson Nano hoặc Mini PC công nghiệp) có thể giúp cải thiện thêm FPS mà không thay đổi kiến trúc scheduling tổng thể.

3.3. Kết luận chương

Trong Chương 3, hệ thống nhận diện đa mô hình đã được thiết kế với các phần chính: nhận diện vật thể, phát hiện làn đường và giám sát trạng thái mắt người lái. Các chức năng được tách thành các tiến trình riêng, chạy song song để tăng tốc xử lý và giảm độ trễ. Dữ liệu camera được chia sẻ giữa các tiến trình qua queue.

Hệ thống hoạt động ổn định đồng thời tối ưu chi phí triển khai. Thiết kế này đảm bảo các chức năng được xử lý hiệu quả, tốc độ đủ nhanh và chuẩn bị sẵn sàng cho thử nghiệm thực tế trong Chương 4.

Chương 4: KẾT QUẢ MÔ PHỎNG VÀ ĐÁNH GIÁ

4.1. Đánh giá hiệu năng từng phần

4.1.1. Môi trường thực nghiệm và cấu hình phần cứng

Các thử nghiệm mô hình nhận diện vật thể được thực hiện trong môi trường mô phỏng và thực tế được mô tả ở bảng 4.1

Bảng 4.1 Môi trường thực nghiệm.

	Nhận diện vật thể/làn đường	Nhận diện trạng thái mắt
Điều kiện thử nghiệm	Sử dụng khu vực thử nghiệm với làn đường được đánh dấu rõ ràng.	Khuôn mặt người điều khiển phương tiện
Điều kiện ánh sáng	Tốt	Tất cả
Phạm vi nhận diện	Các vật thể chính (ô tô, người đi bộ, xe máy) trong khoảng 10m, phù hợp với vùng cảnh báo thực tế cho người lái.	Nhỏ hơn 1m
Phần cứng mô phỏng	ACER Nitro 5 2021 AN515-45 – AMD Ryzen 5 5600H GTX 1650 (Không dùng GPU)	

Với việc không sử dụng GPU thêm vào đó không sử dụng hết công năng của laptop nên chi phí ước tính khoảng 8-10 triệu VND.

Trong mỗi phần sẽ tập trung thảo luận chính vào FPS của mỗi tác vụ. Độ chính xác của các mô hình được tổng hợp ở phần đánh giá hiệu năng tổng thể.

4.1.2. Mô hình nhận diện vật thể

Trong điều kiện ánh sáng ban ngày và tầm nhìn tốt, mô hình nhận diện chính xác các đối tượng chính như người đi bộ, ô tô, xe máy và các vật cản lớn trong phạm vi 0–10m. Tỉ lệ phát hiện đúng (precision) đạt mức 90-95% với các đối tượng rõ ràng. Tỉ lệ nhận diện sai (false detection) thấp khi đối tượng không bị che khuất nặng. Trong môi trường ánh sáng yếu, độ chính xác giảm nhẹ xuống khoảng 75–80%, nhưng vẫn đủ để cảnh báo người lái.

Hình 4.1 thể hiện kết quả thực nghiệm của mô hình nhận diện vật thể YOLOv3-tiny trên video thực tế. Trong hình, các đối tượng như phương tiện được xác định và bao

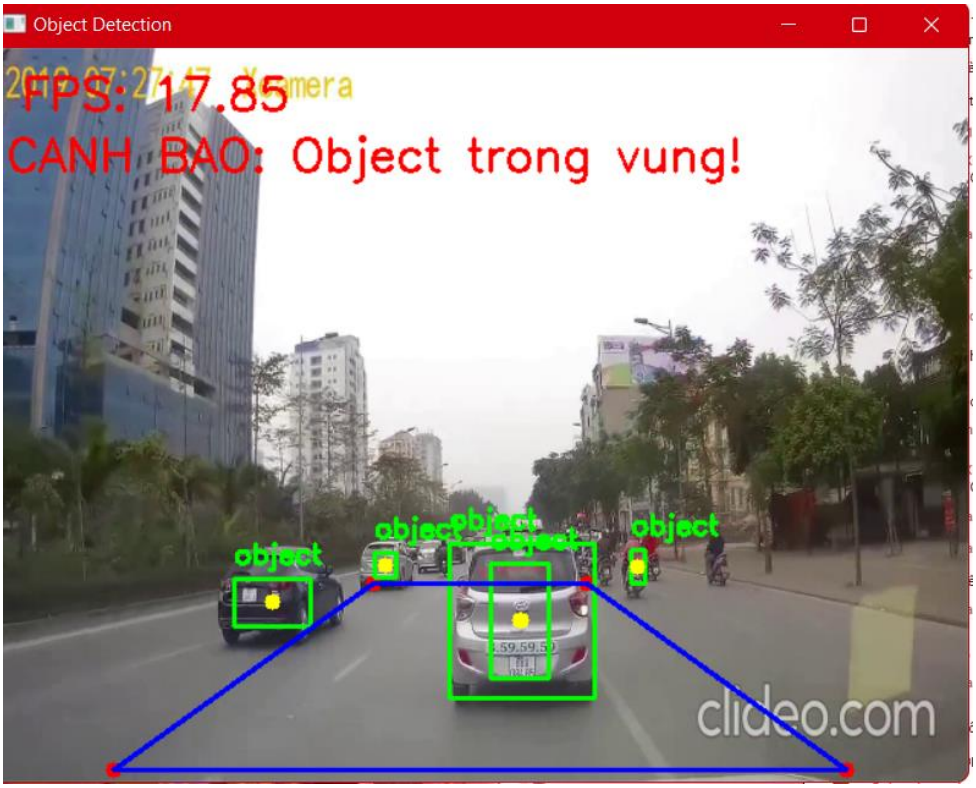
quanh bởi các hộp chữ nhật màu xanh lá, kèm nhãn "object" phía trên. FPS hiện tại được hiển thị ở góc trên bên trái của khung hình, cho thấy tốc độ xử lý của mô hình. Kết quả cho thấy mô hình có khả năng nhận diện chính xác các đối tượng trong phạm vi khoảng 10 mét, phù hợp với mục tiêu hỗ trợ lái xe an toàn.

Hình 4.1 Kết quả thử nghiệm của mô hình nhận diện vật thể.

Hình 4.2 minh họa kết quả thực nghiệm của mô hình YOLOv3 trên cùng một video thử nghiệm. Các đối tượng như phương tiện được nhận diện và bao quanh bằng hộp chữ nhật màu xanh lá kèm nhãn "object". So với YOLOv3-tiny, mô hình YOLOv3 cho độ chính xác cao hơn, nhận diện chi tiết hơn các đối tượng nhỏ, nhưng FPS thấp hơn đáng kể (chỉ khoảng 2 FPS), khiến việc xử lý thời gian thực kém khả thi trong các ứng dụng cần tốc độ cao.

Hình 4.2 Kết quả của mô hình khi dùng Yolov3.

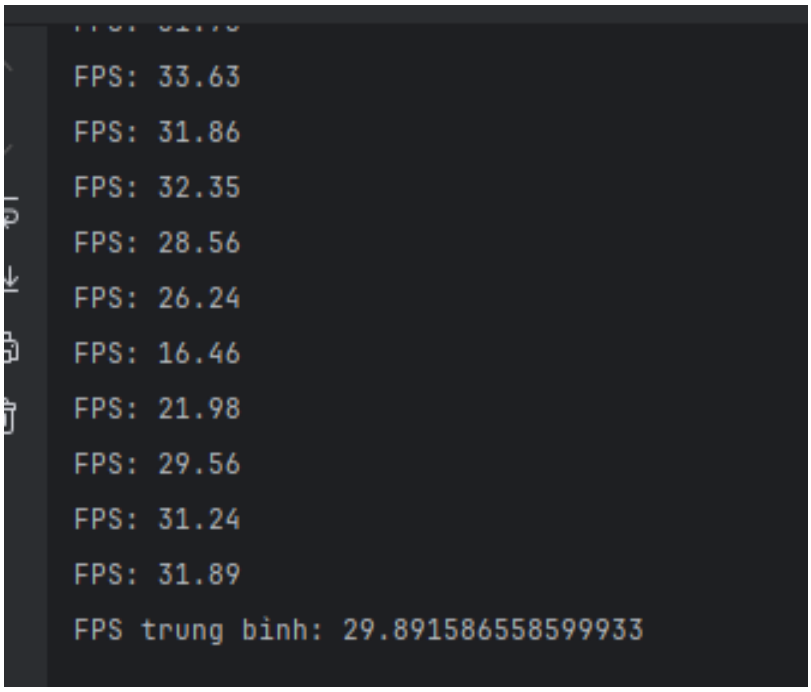
Trong quá trình thực nghiệm, một số vật thể đôi khi bị nhận diện hai lần do thuật toán Non-Maximum Suppression (NMS) chưa loại bỏ hết các dự đoán chồng chéo. Mặc dù có thể giảm ngưỡng NMS để hạn chế hiện tượng này, nhưng việc giảm quá mức sẽ làm các vật thể nhỏ hoặc mờ bị xoá khỏi kết quả, dẫn đến mất thông tin quan trọng, điều này đặc biệt nguy hiểm trong các ứng dụng an toàn giao thông. Do đó, việc chọn ngưỡng NMS là một sự cân bằng giữa hạn chế double detection và đảm bảo không bỏ sót vật thể.



Hình 4.3 Hiện tượng vật thể bị nhận diện nhiều lần.

Tuy nhiên, việc hiển thị hình ảnh trong quá trình mô hình hoạt động làm phát sinh thêm chi phí xử lý, do đó không tối ưu về mặt hiệu suất. Vì lý do này, FPS của hệ thống được đánh giá ở chế độ *inference-only*, tức là chỉ đo thời gian suy luận và hậu xử lý của mô hình mà không thực hiện các bước vẽ và hiển thị kết quả.

Cách đo này giúp phản ánh chính xác hơn hiệu suất tính toán thực tế của mô hình. FPS thực tế của hệ thống được trình bày trong Hình 4.4. FPS trong trường hợp này được cải thiện khoảng 20% so với trường hợp phải hiển thị.



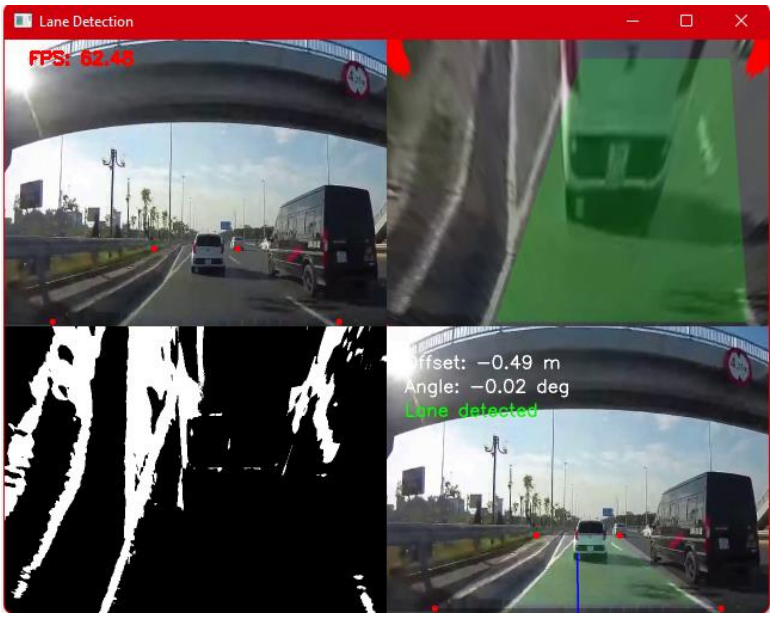
Hình 4.4 FPS thực tế được đo ở chế độ inference-only.

Mặc dù mô hình còn một số hạn chế nhất định, nhưng với việc tối ưu hóa hệ thống ADAS về chi phí và thuật toán, kết quả nhận diện vật thể đáp ứng đúng mục tiêu đề ra. Như vậy, mô hình hoạt động ổn định, chính xác và phù hợp với yêu cầu ứng dụng.

4.1.3. Mô hình nhận diện làn đường

Trong quá trình thực nghiệm, mô hình nhận diện làn đường được đánh giá thông qua một hình ảnh tổng hợp từ bốn bước xử lý được mô tả như trong hình 4.5:

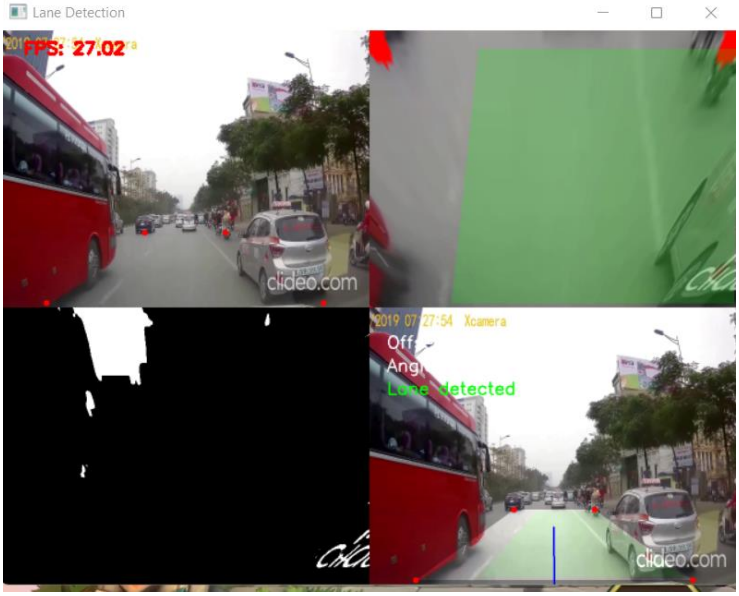
- Ảnh gốc: Đây là hình ảnh thu từ camera, chứa toàn bộ cảnh vật phía trước xe, bao gồm các làn đường, xe di chuyển cùng chiều và các vật thể xung quanh.
- Biến đổi góc nhìn (Perspective Transform / Bird-Eye View): Hình ảnh được chuyển sang góc nhìn từ trên xuống để thuận tiện trong việc xác định vị trí làn đường, giảm méo hình do góc camera.
- Hình nhị phân (Binary Mask): Ảnh sau khi chuyển góc nhìn được xử lý threshold HSV để phân tách làn đường (màu sáng) và nền đường (màu tối), tạo ra hình nhị phân giúp nhận diện dễ dàng hơn.
- Kết quả nhận diện làn đường: Làn đường được vẽ lên ảnh gốc, hiển thị vùng lane, đường trung tâm, offset so với vị trí xe, góc lái tính toán và FPS thực thi của hệ thống.



Hình 4.5 Kết quả nhận diện làn đường.

Mô hình có khả năng xác định làn đường chính xác trong hầu hết các điều kiện đường thẳng và cong nhẹ. Hệ thống tính toán được offset, góc lái và FPS theo thời gian thực, giúp người lái hoặc hệ thống ADAS điều chỉnh hướng đi chính xác.

Hiện tượng dao động của offset và góc lái giữa các frame liên tiếp chủ yếu xuất phát từ sự thay đổi nhỏ của lanemasks khi FPS cao. Giải pháp được đề xuất là lọc các tham số hình học của đường làn (hệ số đa thức hoặc góc lái) theo miền thời gian bằng bộ lọc thông thấp, đồng thời giới hạn tốc độ thay đổi tối đa giữa hai frame liên tiếp. Cách tiếp cận này giúp tăng độ ổn định của làn đường phát hiện mà không làm giảm đáng kể tính phản ứng của hệ thống.

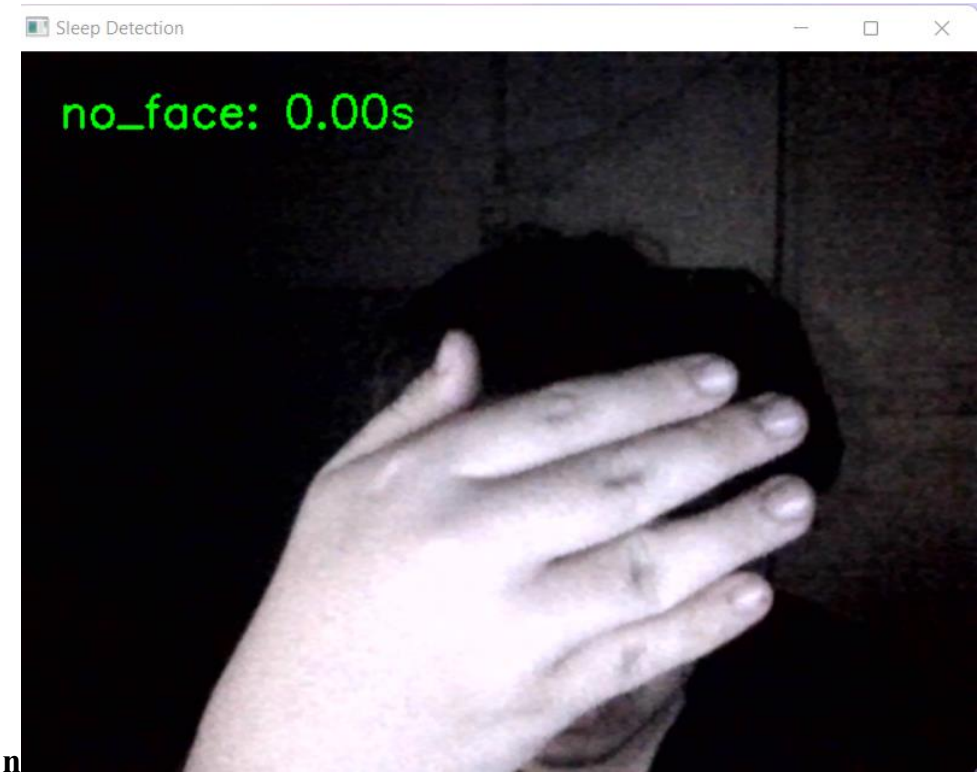


Hình 4.6 Mô hình hoạt động không tốt trong môi trường nhiều phương tiện.

Một trường hợp khác được mô tả trong hình 4.6. Vật thể chắn làn đường, như xe cùng chiều, biển báo hoặc vạch sơn mờ, có thể gây nhầm lẫn tạm thời cho thuật toán. Để cải thiện, có thể kết hợp các bộ lọc offset/góc lái, tích hợp camera phụ hoặc sensor bổ sung.

4.1.4. Mô hình nhận diện trạng thái mắt

Hệ thống nhận diện trạng thái mắt sử dụng mô hình MediaPipe FaceMesh đã được thử nghiệm trên nhiều trường hợp khác nhau để đánh giá khả năng phát hiện mắt mở và mắt nhắm, cũng như xử lý các tình huống đặc biệt khi khuôn mặt bị che khuất.



Hình 4.8 Trường hợp không nhận biết được khuôn mặt.

Hình 4.8 mô tả trường hợp người dùng dơ tay che mặt, mô hình không thể nhận diện được khuôn mặt và các landmark mắt. Kết quả này cho thấy hạn chế của hệ thống trong các tình huống khuôn mặt bị che hoặc bị che khuất bởi vật thể khác, khi đó hệ thống báo trạng thái là “no_face”. Hình ảnh minh họa rõ ràng phản ánh việc mắt không được phát hiện khi khuôn mặt bị che, đồng thời nhấn mạnh tầm quan trọng của việc đảm bảo tầm nhìn trực diện để hệ thống hoạt động chính xác.

Hình 4.9 là kết quả mô phỏng trong trường hợp mắt mở và mắt nhắm. Mô hình nhận diện trạng thái mắt chính xác trong cả hai trường hợp, khi mắt mở hệ thống báo “open”, khi mắt nhắm hệ thống báo “closed” và bắt đầu đếm thời gian nhắm mắt liên tục. Kết quả này minh họa khả năng phát hiện chính xác các trạng thái mắt, cho phép hệ thống theo dõi mức độ mệt mỏi hoặc buồn ngủ của người lái trong thời gian thực.

Hình 4.9 Trường hợp nhắm mắt và mở mắt.

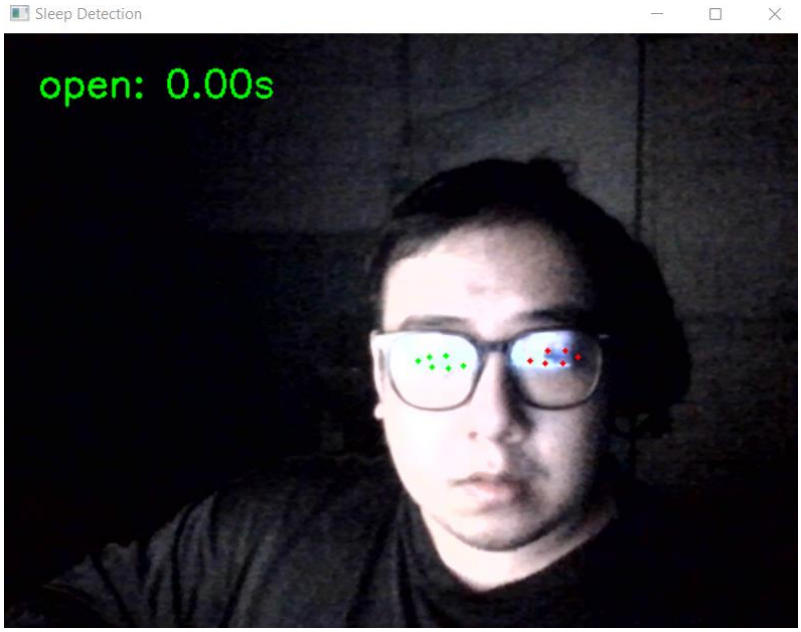
Trong quá trình thử nghiệm, hệ thống nhận diện mắt cũng được đánh giá khi người dùng đeo kính hoặc khi ánh sáng chiếu trực tiếp vào mắt như hình 4.10. Kết quả cho thấy ánh sáng mạnh hoặc phản chiếu trên bề mặt kính có thể làm các landmark mắt trở nên khó nhận diện, dẫn đến hệ thống báo "no_face" hoặc không xác định đúng trạng thái mắt (mắt mở/nhắm).

Hiện tượng này xảy ra do sự thay đổi cường độ sáng và tương phản trên vùng mắt, làm mô hình MediaPipe FaceMesh khó phân biệt chính xác các điểm landmark quan trọng. Tình huống này phản ánh một hạn chế thực tế của hệ thống khi triển khai ngoài môi trường lý tưởng.

Để cải thiện, có thể kết hợp các giải pháp như:

- Điều chỉnh cường độ ánh sáng hoặc góc camera để giảm chói.
- Sử dụng các thuật toán tiền xử lý hình ảnh, như cân bằng sáng (histogram equalization) hoặc lọc gamma, để giảm ảnh hưởng phản chiếu trên kính.
- Kết hợp thêm nhiều sensor (ví dụ hồng ngoại) để phát hiện mắt trong điều kiện ánh sáng khó khăn.

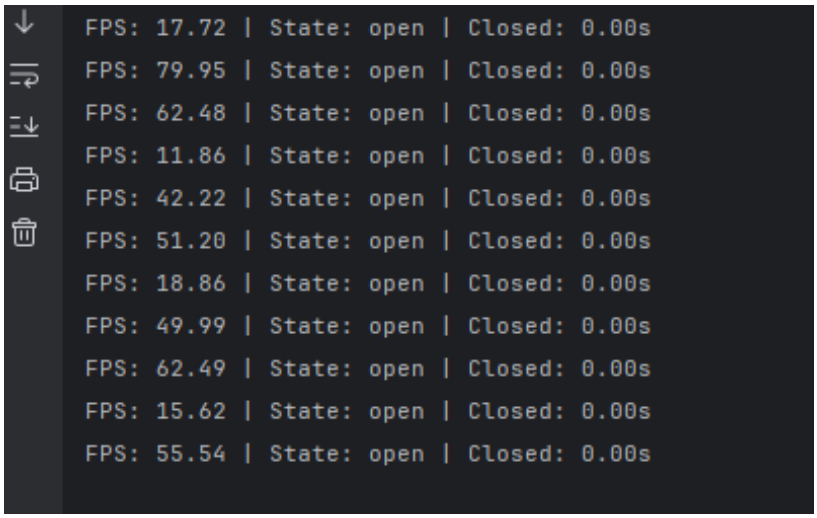
Kết quả này nhấn mạnh tầm quan trọng của việc tối ưu hóa cả phần cứng và phần mềm để hệ thống nhận diện mắt hoạt động ổn định trong nhiều điều kiện thực tế.



Hình 4.10 Độ tương phản làm ảnh hưởng khả năng dự đoán của mô hình.

Tổng quan, các kết quả thử nghiệm cho thấy mô hình hoạt động tốt trong các điều kiện thuận lợi, đồng thời cung cấp thông tin quan trọng về hạn chế khi khuôn mặt bị che. Những kết quả này là cơ sở để tiếp tục tối ưu hóa thuật toán, cải thiện độ bền vững và khả năng nhận diện trong các tình huống thực tế khác nhau.

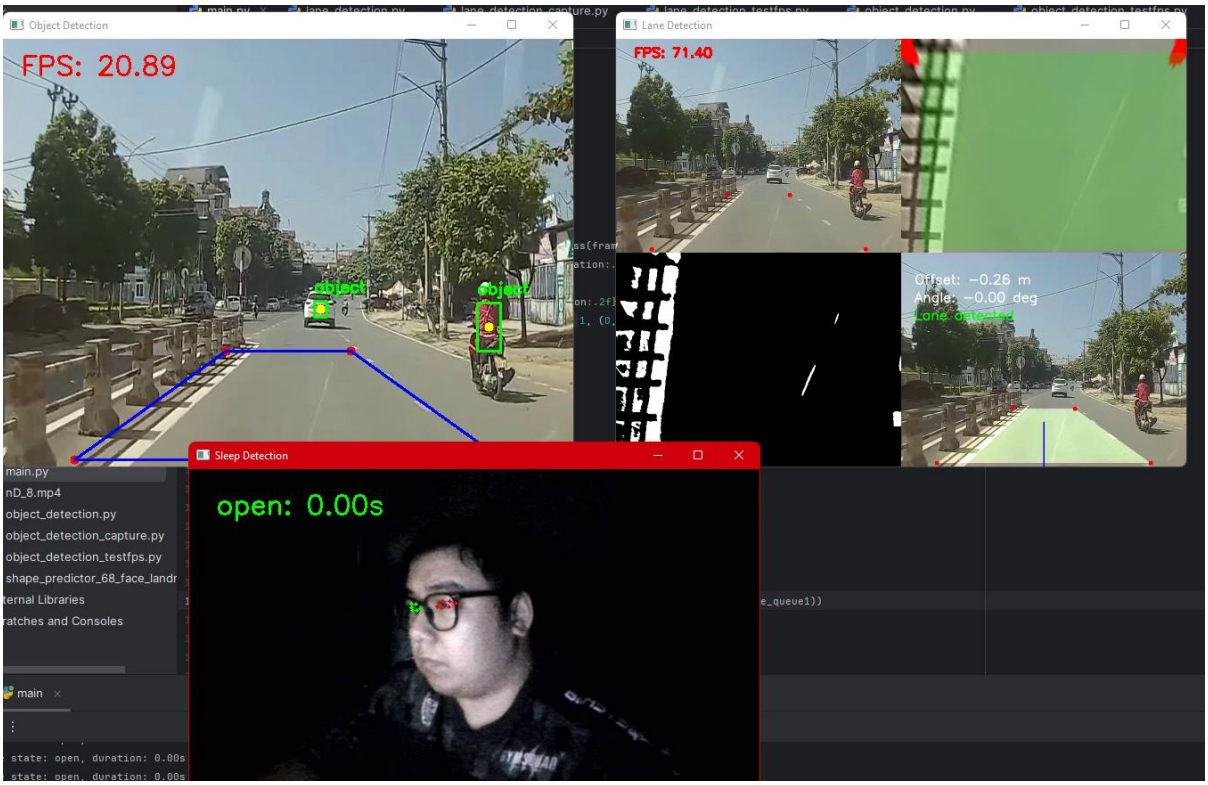
Mô hình nhận diện trạng thái mắt cho kết quả FPS tốt (Hình 4.11). Tuy nhiên, do trạng thái nhắm–mở của mắt người không thay đổi nhanh, việc sử dụng mô hình có hiệu suất cao như mô hình này là chưa thật sự tối ưu về mặt tài nguyên tính toán. Trong tương lai, có thể thay thế bằng các mô hình nhẹ hơn nhằm tối ưu hiệu năng hệ thống.



Hình 4.11 FPS khi hoạt động trên thực tế.

4.2. Đánh giá hiệu năng tổng thể

Hệ thống được thiết kế chạy đồng thời ba mô hình: nhận diện trạng thái mắt, nhận diện làn đường, và nhận diện vật. Các tiến trình được tách riêng và giao tiếp qua queue, đảm bảo việc xử lý song song và độ ổn định cao (Hình 4.12).



Hình 4.12 Kết quả khi chạy một lần ba mô hình nhận diện.

Bảng 4.2 trình bày kết quả đánh giá hiệu năng thực nghiệm của các mô-đun chính trong hệ thống ADAS, bao gồm nhận diện vật thể, nhận diện làn đường và giám sát trạng thái mắt người lái. Việc đánh giá được thực hiện trên các tập mẫu kiểm tra khác nhau, phù hợp với đặc điểm và thời gian thử nghiệm của từng mô-đun. Mô hình được đánh giá không sử dụng chế độ inference-only. Hiệu suất thực tế của các mô-đun có thể được cải thiện 20-30% như đã bàn luận ở phía trên.

Bảng 4.2 Đánh giá hiệu năng tổng thể của hệ thống.

	Nhận diện vật thể	Nhận diện làn đường	Nhận diện trạng thái mắt
Số mẫu kiểm tra	405	234	103
Độ chính xác	91%	76.92%	97%
FPS trung bình	20FPS	60FPS	60FPS

Đối với mô-đun nhận diện vật thể, hệ thống đạt độ chính xác khoảng 91% trên 405 mẫu kiểm tra, với tốc độ xử lý trung bình khoảng 20 FPS. Kết quả này cho thấy mô hình YOLOv3-tiny đáp ứng tốt yêu cầu nhận diện trong điều kiện thời gian thực mềm trên nền tảng CPU-only, đồng thời đảm bảo cân bằng giữa độ chính xác và tốc độ xử lý.

Mô-đun nhận diện làn đường đạt độ chính xác khoảng 76.92% trên 234 mẫu kiểm tra, với tốc độ xử lý trung bình lên đến 60 FPS. Độ chính xác của mô-đun này chịu ảnh hưởng đáng kể bởi điều kiện ánh sáng, chất lượng vạch kẻ đường và hình dạng tuyến đường. Tuy nhiên, nhờ sử dụng các thuật toán xử lý ảnh truyền thống có độ phức tạp

thấp, mô-đun vẫn đảm bảo tốc độ xử lý cao và ổn định, phù hợp cho chức năng cảnh báo lệch làn đường.

Đối với mô-đun giám sát trạng thái mắt người lái, hệ thống đạt độ chính xác khoảng 97% trên 103 mẫu kiểm tra, với tốc độ xử lý trung bình khoảng 60 FPS. Kết quả này cho thấy phương pháp sử dụng MediaPipe kết hợp chỉ số EAR hoạt động hiệu quả trong điều kiện thử nghiệm ổn định, đáp ứng tốt yêu cầu phát hiện trạng thái buồn ngủ của người lái.

Nhìn chung, các kết quả trong Bảng 4.2 cho thấy hệ thống ADAS đề xuất có khả năng hoạt động ổn định và đáp ứng yêu cầu thời gian thực mềm. Mặc dù độ chính xác của từng mô-đun có sự khác nhau do đặc thù bài toán và điều kiện thử nghiệm, nhưng tổng thể hệ thống vẫn đảm bảo hiệu năng cần thiết cho mục tiêu cảnh báo và hỗ trợ người lái trong phạm vi đề tài. Tuy nhiên, hệ thống vẫn còn một số hạn chế ở từng phần riêng lẻ đã được đề cập ở phần trên như:

- Nhận diện mắt có thể bị ảnh hưởng bởi ánh sáng mạnh hoặc kính.
- Làn đường có dao động khi FPS cao hoặc vật thể chắn, nhưng vẫn cung cấp thông tin kịp thời để cảnh báo.
- Các vật thể nhỏ có thể bị bỏ sót nếu NMS threshold được giảm để tránh trùng lặp.
- Trên lý thuyết việc sử dụng process giúp tối ưu thiết kế, nhưng việc làm quá tải cho CPU có thể khiến suy giảm hiệu suất của mô hình.

4.3. Kết luận chương

Trong Chương 4, đề tài đã tiến hành mô phỏng và đánh giá hiệu năng của từng mô-đun trong hệ thống hỗ trợ lái xe, bao gồm nhận diện vật thể, nhận diện làn đường và giám sát trạng thái mắt người lái. Các thử nghiệm được thực hiện trong môi trường mô phỏng và điều kiện thực tế nhằm kiểm tra tính khả thi của hệ thống khi triển khai trên nền tảng phần cứng phổ thông, không sử dụng GPU tăng tốc.

Kết quả thực nghiệm cho thấy mô hình nhận diện vật thể sử dụng YOLOv3-tiny có khả năng phát hiện chính xác các đối tượng chính trong phạm vi mục tiêu, đồng thời đạt tốc độ xử lý phù hợp với yêu cầu thời gian thực.

Đối với mô hình nhận diện làn đường, thuật toán dựa trên các kỹ thuật xử lý ảnh truyền thống cho kết quả ổn định trong điều kiện đường rõ ràng và ánh sáng thuận lợi. Hệ thống có thể ước lượng được độ lệch làn và góc lái theo thời gian thực, phục vụ cho chức năng cảnh báo lệch làn.

Mô hình nhận diện trạng thái mắt người lái cho kết quả phân loại chính xác trong các điều kiện thử nghiệm thông thường và đạt FPS tốt.

Tổng thể, các kết quả trong chương này cho thấy hệ thống đề xuất hoạt động ổn định, đáp ứng được các yêu cầu cơ bản của một hệ thống hỗ trợ lái xe thời gian thực mềm, đồng thời làm rõ những hạn chế còn tồn tại để làm cơ sở cho các hướng cải tiến và phát triển tiếp theo.

Chương 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

Trong báo cáo này, đề tài đã trình bày chi tiết về thiết kế và triển khai hệ thống nhận diện đa mô hình hỗ trợ lái xe, bao gồm các thành phần chính: nhận diện làn đường (Lane Detection), nhận diện vật thể (Object Detection) và giám sát trạng thái mắt người lái (Eye Sleep Detection). Các mô hình AI được lựa chọn dựa trên tiêu chí cân bằng giữa hiệu suất, độ chính xác, chi phí và khả năng xử lý thời gian thực.

Cụ thể, YOLOv3-tiny được sử dụng cho nhận diện vật thể nhằm tối ưu FPS, MediaPipe FaceMesh được áp dụng cho giám sát trạng thái mắt và phương pháp histogram kết hợp sliding window được sử dụng cho nhận diện làn đường.

Kết quả thiết kế cho thấy hệ thống có thể hoạt động ổn định trong môi trường thực nghiệm, nhận diện chính xác các vật thể trong phạm vi 10m – phù hợp với mục tiêu cảnh báo sớm và hỗ trợ người lái. Tốc độ xử lý của mô hình YOLOv3-tiny đạt khoảng 20 FPS, đủ để đáp ứng yêu cầu thời gian thực trong các tình huống giao thông thông thường.

Hệ thống có khả năng đánh giá vị trí và khoảng cách tương đối của làn đường, đồng thời cảnh báo sớm khi có nguy cơ va chạm hoặc lệch làn, qua đó nâng cao độ an toàn cho phương tiện.

Ngoài ra hệ thống có khả năng giám sát trạng thái người lái xe, giúp cảnh báo khả năng khi người lái có dấu hiện buồn ngủ trong khi lái.

5.2. Hạn chế

Mặc dù hệ thống hoạt động tốt, nhưng vẫn tồn tại một số hạn chế đáng chú ý:

- Hệ thống nhận diện bị ảnh hưởng bởi các điều kiện ánh sáng phức tạp như chói, ánh sáng yếu, thời tiết mưa hoặc sương mù. Những yếu tố này có thể làm giảm độ chính xác của cả nhận diện vật thể và trạng thái mắt.
- Khi sử dụng YOLOv3 gốc, FPS chỉ đạt 1-2, không đáp ứng được nhu cầu thời gian thực. Việc giảm NMS threshold để tăng độ chính xác có thể dẫn đến vật thể nhỏ bị loại bỏ, ảnh hưởng đến an toàn. YOLOv3-tiny cải thiện tốc độ nhưng vẫn còn hạn chế trong môi trường nhiều đối tượng đồng thời.
- Hệ thống hiện tại chủ yếu giám sát làn đường, vật thể và trạng thái mắt phía trước xe. Chưa có giải pháp giám sát toàn diện cho các tình huống phức tạp, như nhiều phương tiện di chuyển cùng lúc, người đi bộ bất ngờ xuất hiện hay các tình huống giao thông ngoài dự đoán.

- Việc tối ưu chi phí là ưu tiên, tuy nhiên các giải pháp phần cứng tiết kiệm có thể hạn chế khả năng mở rộng hoặc xử lý thêm nhiều mô hình cùng lúc.

5.3. Hướng phát triển

Nhằm cải thiện hiệu quả hệ thống mà không làm tăng quá nhiều chi phí hoặc phức tạp phần cứng, các hướng phát triển đơn giản bao gồm:

- Điều chỉnh các tham số của YOLOv3-tiny, MediaPipe FaceMesh và thuật toán lane detection để tăng độ chính xác và FPS mà không cần thay đổi phần cứng.
- Thêm các bước tiền xử lý hình ảnh như cân bằng sáng tự động, lọc nhiễu, để hệ thống hoạt động ổn định hơn trong các điều kiện ánh sáng khác nhau.
- Tích hợp cảnh báo âm thanh hoặc giao diện trực quan đơn giản khi phát hiện nguy cơ lệch làn hoặc vật thể gần.
- Hiển thị thông tin về làn đường, vật thể và trạng thái mắt trên màn hình xe hoặc laptop, giúp người lái dễ theo dõi.

Đối với tầm nhìn dài hạn, nhằm nâng cao khả năng nhận diện và mở rộng phạm vi ứng dụng, các hướng phát triển xa gồm:

- Sử dụng camera đa hướng, LiDAR, radar hoặc cảm biến siêu âm để giám sát toàn diện môi trường xung quanh xe, tăng độ chính xác và an toàn.
- Áp dụng các mô hình mới hơn như YOLOv8, các kiến trúc Transformer nhẹ hoặc mô hình học sâu đa nhiệm để tăng độ chính xác đồng thời duy trì tốc độ xử lý cao.
- Nhận diện biển báo, người đi bộ, xe máy và các vật thể bất ngờ xuất hiện; đồng thời phát hiện các tình huống phức tạp như tắc đường, thời tiết xấu.
- Tích hợp cảnh báo chủ động kết hợp âm thanh, rung ghế hoặc hiển thị AR để người lái phản ứng kịp thời.
- Cho phép mô hình tự điều chỉnh theo điều kiện lái thực tế và phong cách lái của người dùng, tăng khả năng ứng dụng trong nhiều môi trường khác nhau.

5.4. Đề xuất phần cứng

5.4.1. Bộ xử lý và điều khiển trung tâm

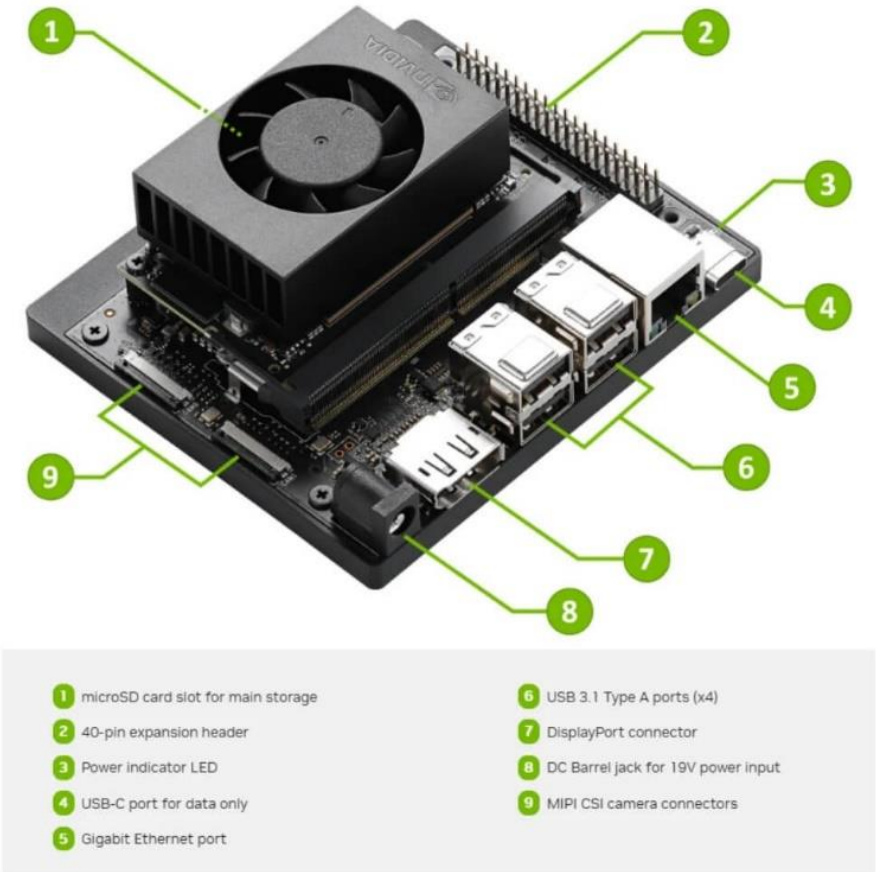
Trong định hướng phát triển tiếp theo, nền tảng NVIDIA Jetson Nano được đề xuất làm phần cứng nhúng để triển khai hệ thống ADAS. Jetson Nano là một nền tảng SoC tích hợp CPU ARM đa nhân và GPU CUDA, được thiết kế chuyên cho các ứng dụng trí tuệ nhân tạo nhúng với mức tiêu thụ năng lượng thấp.

Jetson Nano đáp ứng tốt các yêu cầu của hệ thống đề tài, bao gồm:

- Khả năng xử lý song song nhiều tác vụ nhờ CPU đa nhân,
- Hỗ trợ tăng tốc suy luận mô hình học sâu thông qua GPU CUDA và các thư viện tối ưu như TensorRT,

- Khả năng chạy các mô hình nhẹ như YOLOv3-tiny ở chế độ thời gian thực mềm,
- Chi phí phần cứng ở mức hợp lý (8-10tr), phù hợp với mục tiêu tối ưu chi phí của hệ thống ADAS chi phí thấp.

Ngoài ra, môi trường phát triển trên Jetson Nano hỗ trợ tốt Python, OpenCV và các framework AI phổ biến, giúp việc chuyển đổi từ môi trường mô phỏng trên máy tính cá nhân sang nền tảng nhúng không yêu cầu thay đổi lớn về kiến trúc phần mềm.



Hình 5.1 NVIDIA AI Jetson Nano

5.4.2. Cảm biến tích hợp

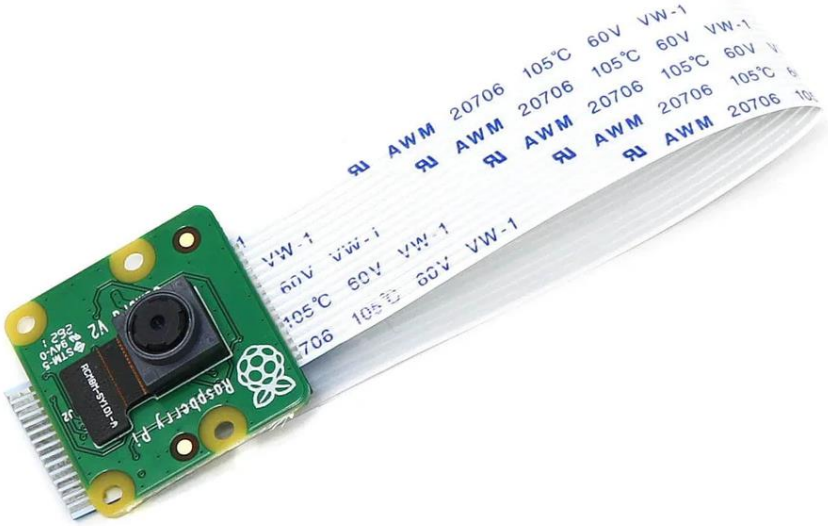
5.4.2.1. Camera

Camera đóng vai trò là cảm biến chính trong hệ thống, cung cấp dữ liệu hình ảnh cho các mô-đun nhận diện. Camera được sử dụng để:

- Nhận diện vật thể phía trước,
- Phát hiện làn đường,
- Giám sát trạng thái mắt người lái.

Việc sử dụng camera đơn giúp giảm chi phí phần cứng và đơn giản hóa hệ thống, phù hợp với mục tiêu mô phỏng ADAS chi phí thấp.

Hình 5.2 mô tả module camera Raspberry Pi Camera Module V2 sử dụng cảm biến Sony IMX219 với độ phân giải 8 MP, giao tiếp CSI. Camera cho độ trễ thấp và chất lượng hình ảnh ổn định, phù hợp làm cảm biến hình ảnh chính trong hệ thống ADAS.



Hình 5.2 Raspberry Pi Camera Module V2 8MP

5.4.2.2. Cảm biến siêu âm (Ultrasonic Sensor - US)

Cảm biến siêu âm có thể được tích hợp để đo khoảng cách ngắn phía trước hoặc phía sau xe. Cảm biến này có ưu điểm:

- Chi phí thấp,
- Hoạt động ổn định trong điều kiện ánh sáng kém,
- Bổ sung thông tin khoảng cách cho hệ thống cảnh báo va chạm ở cự ly gần.

Dữ liệu từ cảm biến siêu âm có thể được kết hợp với kết quả nhận diện từ camera để tăng độ tin cậy của cảnh báo.

Hình 5.3 Cảm biến siêu âm HC-R704

Hình 5.3 cảm biến siêu âm HC-SR04 có khả năng đo khoảng cách trong dải 2–400 cm, hoạt động ổn định trong điều kiện ánh sáng kém và được sử dụng để bổ trợ cảnh báo va chạm cự ly gần.

5.4.2.3. *Cảm biến quán tính (Inertial Measurement Unit - IMU)*

Cảm biến IMU (gia tốc kế và con quay hồi chuyển) được đề xuất để thu thập thông tin về:

- Gia tốc và rung động của xe,
- Hướng chuyển động và thay đổi quỹ đạo.

IMU hỗ trợ ổn định thuật toán nhận diện làn đường và giúp phân biệt giữa chuyển động thực của xe và nhiễu từ môi trường, đặc biệt khi xe di chuyển trên đường không bằng phẳng.

Hình 5.4 GY-9250 9DOF IMU MPU9250

Cảm biến quán tính MPU-9250 tích hợp gia tốc kế và con quay hồi chuyển, cung cấp thông tin chuyển động nhằm hỗ trợ ổn định thuật toán nhận diện làn đường (Hình 5.4).

5.4.2.4. Cảm biến tốc độ (Speed Sensor - SS)

Thông tin tốc độ xe có thể được thu thập thông qua cảm biến tốc độ bánh xe hoặc giao diện OBD-II. Dữ liệu này cho phép:

- Điều chỉnh ngưỡng cảnh báo phù hợp với tốc độ,
- Hỗ trợ logic cảnh báo va chạm và lệch làn.

Việc tích hợp dữ liệu tốc độ giúp hệ thống ADAS đưa ra cảnh báo phù hợp hơn với tình trạng vận hành thực tế.

Hình 5.5 là thiết bị OBD-II ELM327 cho phép thu thập dữ liệu vận hành của xe như tốc độ và vòng tua động cơ, hỗ trợ điều chỉnh ngưỡng cảnh báo theo điều kiện vận hành thực tế.

Hình 5.5 Cảm biến tốc độ ELM327 USB OBD2

TÀI LIỆU THAM KHẢO

[1] Toyota Việt Nam, "ADAS là gì? Tìm hiểu tính năng hỗ trợ lái xe ô tô ADAS," 2024. [Online]. Available: https://www.toyota.com.vn/tin-tuc/thong-tin-bo-tro/he-thong-adas-40143

[2] Tinhte.vn, "Vì sao xe hơi có hệ thống phanh khẩn cấp chủ động vẫn xảy ra tai nạn." [Online]. Available: https://tinhte.vn/thread/vi-sao-xe-hoi-co-he-thong-phanh-khan-cap-chu-dong-van-xay-ra-tai-nan.3555520/

[3] VIETMAP, "Hệ thống cảnh báo lệch làn đường có tác dụng gì?," 2023. [Online]. Available: https://vietmap.vn/he-thong-canh-bao-lech-lan-duong-co-tac-dung-gi

[4] Tinhte.vn, "Hệ thống nhận diện người đi bộ dành cho xe tự lái với độ chính xác cao gần như theo thời gian thực."[Online]. Available: https://tinhte.vn/thread/he-thong-nhan-dien-nguoi-di-bo-danh-cho-xe-tu-lai-voi-do-chinh-xac-gan-nhu-theo-thoi-gian-thuc.2550064/

[5] V. Savitha et al., "Advance Driver Assistance System," *International Journal of Novel Research and Development*, vol. 9, no. 3, 2024.

[6] L. A. Tran, T. D. Do, D. C. Park, and M. H. Le, "Robustness Enhancement of Object Detection in Advanced Driver Assistance Systems (ADAS)," *arXiv preprint*, 2021.

[7] Ultralytics, "YOLOv3 and YOLOv3u Documentation," 2025. [Online]. Available: https://docs.ultralytics.com/vi/models/yolov3/

[8] A. G. Sawant et al., "A Real-Time Driver Drowsiness Detection System Using MediaPipe and Eye Aspect Ratio," *arXiv preprint*, 2025.

[9] C. M. Anumba and F. Chan, *Intelligent Vehicles and Advanced Driver Assistance Systems*. Springer, 2020.

[10] Wikipedia, "Advanced driver-assistance system," 2025.[Online]. Available: https://en.wikipedia.org/wiki/Advanced_driver-assistance_system

[11] A. Esquer, "TBR Technical Corner: Braking Requirements for Optimizing Autonomous Emergency Braking (AEB) Performance (Part 1 of 3)," *The Brake Report*, Oct. 20, 2020. [Online]. Available: https://thebrakereport.com/tbr-technical-corner-braking-requirements-for-optimizing-autonomous-emergency-braking-aeb-performance-part-1-out-of-3/?utm_source=chatgpt.com

[12] J. He, H. Wang, and B. Zhang, "A Review of Lane Detection Methods Based on Computer Vision and Deep Learning," Pattern Recognition, Elsevier, 2020.

[13] Wikipedia, "Hough Transform," 2025. [Online]. Available: https://en.wikipedia.org/wiki/Hough_transform

[14] J. Canny, "A Computational Approach to Edge Detection," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 8, no. 6, pp. 679–698, 1986.

PHỤ LỤC

1. Object Detection

```
import cv2
import numpy as np
import time

class ObjectDetector:
    def __init__(self, net, output_size=(640, 480), conf_threshold=0.5,
nms_threshold=0.3):
        self.net = net
        self.output_size = output_size
        self.conf_threshold = conf_threshold
        self.nms_threshold = nms_threshold
        layer_names = self.net.getLayerNames()
        self.output_layers = [layer_names[i - 1] for i in
self.net.getUnconnectedOutLayers()]

    def detect_objects(self, frame):
        """
        Nhận frame, trả về frame đã vẽ bbox và FPS
        """
        start_time = time.time()
        frame_resized = cv2.resize(frame, self.output_size)
        height, width = frame_resized.shape[:2]

        # Vẽ các điểm góc
        tl = (250, 350)
        tr = (390, 350)
        bl = (80, 472)
        br = (560, 472)
        for pt in [tl, tr, bl, br]:
            cv2.circle(frame_resized, pt, 5, (0, 0, 255), -1)

        # Tạo blob và forward
        blob = cv2.dnn.blobFromImage(frame_resized, 1/255.0, (416, 416),
swapRB=True, crop=False)
        self.net.setInput(blob)
        outs = self.net.forward(self.output_layers)

        class_ids = []
        confidences = []
        boxes = []

        for out in outs:
            for detection in out:
                scores = detection[5:]
                class_id = np.argmax(scores)
                confidence = scores[class_id]
```

```
                if confidence > self.conf_threshold:
                    center_x = int(detection[0] * width)
                    center_y = int(detection[1] * height)
                    w = int(detection[2] * width)
                    h = int(detection[3] * height)
                    x = max(0, center_x - w // 2)
                    y = max(0, center_y - h // 2)
                    w = min(w, width - x)
                    h = min(h, height - y)

                    boxes.append([x, y, w, h])
                    confidences.append(float(confidence))
                    class_ids.append(class_id)

        # Non-max suppression
        indexes = cv2.dnn.NMSBoxes(boxes, confidences, self.conf_threshold,
self.nms_threshold)
        final_boxes = []
        if len(indexes) > 0:
            for i in indexes.flatten():
                x, y, w, h = boxes[i]
                final_boxes.append([x, y, w, h])
                label = "object"
                cv2.rectangle(frame_resized, (x, y), (x + w, y + h), (0, 255, 0),
2)
                cv2.putText(frame_resized, label, (x, y - 10),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

        fps = 1.0 / (time.time() - start_time)
        cv2.putText(frame_resized, f"FPS: {fps:.2f}", (20, 40),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)
        self.last_boxes = final_boxes
        return frame_resized, fps, final_boxes
```

2. Object Detection (inference-only)

```
import cv2
import numpy as np
import time

class ObjectDetector:
    def __init__(self, net, input_size=(416, 416), conf_threshold=0.5,
nms_threshold=0.3):
        self.net = net
        self.input_size = input_size
        self.conf_threshold = conf_threshold
        self.nms_threshold = nms_threshold

        layer_names = self.net.getLayerNames()
        self.output_layers = [layer_names[i - 1] for i in
self.net.getUnconnectedOutLayers()]

    def detect_fps(self, frame):
        """
```

```
        Nhận frame, chỉ trả về FPS
        """
        start_time = time.time()
        height, width = frame.shape[:2]

        # Tạo blob
        blob = cv2.dnn.blobFromImage(
            frame,
            scalefactor=1 / 255.0,
            size=self.input_size,
            swapRB=True,
            crop=False
        )

        self.net.setInput(blob)
        outs = self.net.forward(self.output_layers)

        boxes = []
        confidences = []

        for out in outs:
            for detection in out:
                scores = detection[5:]
                confidence = scores[np.argmax(scores)]

                if confidence > self.conf_threshold:
                    cx = int(detection[0] * width)
                    cy = int(detection[1] * height)
                    w = int(detection[2] * width)
                    h = int(detection[3] * height)
                    x = cx - w // 2
                    y = cy - h // 2

                    boxes.append([x, y, w, h])
                    confidences.append(float(confidence))

        # NMS (không vẽ)
        if boxes:
            cv2.dnn.NMSBoxes(boxes, confidences,
                             self.conf_threshold,
                             self.nms_threshold)

        fps = 1.0 / (time.time() - start_time)
        return fps

weights = "H:/PBL5/yolov3_tiny/yolov3-tiny.weights"
cfg = "H:/PBL5/yolov3_tiny/yolov3-tiny.cfg"
# Load YOLO
net = cv2.dnn.readNet(weights, cfg)

# Tối ưu OpenCV DNN
net.setPreferableBackend(cv2.dnn.DNN_BACKEND_OPENCV)
net.setPreferableTarget(cv2.dnn.DNN_TARGET_CPU)
```

```
    # Nếu có CUDA:
    # net.setPreferableTarget(cv2.dnn.DNN_TARGET_CUDA)

    detector = ObjectDetector(net)

    cap = cv2.VideoCapture("VID/VN_1.mp4")  # camera

    fps_avg = 0
    count = 0

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        fps = detector.detect_fps(frame)

        fps_avg += fps
        count += 1

        print(f"FPS: {fps:.2f}")

        if cv2.waitKey(1) & 0xFF == 27:  # ESC
            break

    print("FPS trung bình:", fps_avg / max(count, 1))

    cap.release()
    cv2.destroyAllWindows()
```

3. Eyes Detection

```
import cv2
import mediapipe as mp
import time
import math


class EyeSleepDetector:
    def __init__(self, ear_threshold=0.25):
        # MediaPipe FaceMesh
        self.mp_face_mesh = mp.solutions.face_mesh
        self.face_mesh = self.mp_face_mesh.FaceMesh(refine_landmarks=True)

        # Landmark mắt
        self.LEFT_EYE_IDX = [33, 160, 158, 133, 153, 144]
        self.RIGHT_EYE_IDX = [362, 385, 387, 263, 373, 380]

        # Ngưỡng EAR
        self.EYE_AR_THRESH = ear_threshold

        # Biến theo dõi thời gian mắt nhắm
        self.eye_closed_start = None
```

```
    def _eye_aspect_ratio(self, landmarks, eye_idx, frame_w, frame_h):
        coords = [(int(landmarks[i].x * frame_w), int(landmarks[i].y * frame_h))
for i in eye_idx]
        A = math.hypot(coords[1][0] - coords[5][0], coords[1][1] - coords[5][1])
        B = math.hypot(coords[2][0] - coords[4][0], coords[2][1] - coords[4][1])
        C = math.hypot(coords[0][0] - coords[3][0], coords[0][1] - coords[3][1])
        ear = (A + B) / (2.0 * C)
        return ear

    def detect_sleepiness(self, frame):
        eye_closed_duration = 0
        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = self.face_mesh.process(frame_rgb)
        frame_h, frame_w = frame.shape[:2]

        if results.multi_face_landmarks:
            face_landmarks = results.multi_face_landmarks[0]

            # ===== DẪN LANDMARK MẮT =====
            for idx in self.LEFT_EYE_IDX:
                x = int(face_landmarks.landmark[idx].x * frame_w)
                y = int(face_landmarks.landmark[idx].y * frame_h)
                cv2.circle(frame, (x, y), 2, (0, 255, 0), -1)

            for idx in self.RIGHT_EYE_IDX:
                x = int(face_landmarks.landmark[idx].x * frame_w)
                y = int(face_landmarks.landmark[idx].y * frame_h)
                cv2.circle(frame, (x, y), 2, (0, 0, 255), -1)

            # ===== TÍNH EAR =====
            leftEAR = self._eye_aspect_ratio(face_landmarks.landmark,
self.LEFT_EYE_IDX, frame_w, frame_h)
            rightEAR = self._eye_aspect_ratio(face_landmarks.landmark,
self.RIGHT_EYE_IDX, frame_w, frame_h)

            left_closed = leftEAR < self.EYE_AR_THRESH
            right_closed = rightEAR < self.EYE_AR_THRESH

            # Nếu cả 2 mắt nhắm
            if left_closed and right_closed:
                if self.eye_closed_start is None:
                    self.eye_closed_start = time.time()
                eye_closed_duration = time.time() - self.eye_closed_start
                return "closed", eye_closed_duration

            # Nếu mở (hoặc chỉ nhắm 1 mắt)
            else:
                self.eye_closed_start = None
                return "open", 0

        else:
            self.eye_closed_start = None
            return "no_face", 0
```

4. Eyes Detection(inference-only)

```
import cv2
import mediapipe as mp
import time
import math


class EyeSleepDetector:
    def __init__(self, ear_threshold=0.25):
        self.face_mesh = mp.solutions.face_mesh.FaceMesh(
            static_image_mode=False,
            max_num_faces=1,
            refine_landmarks=True,
            min_detection_confidence=0.5,
            min_tracking_confidence=0.5
        )

        self.LEFT_EYE_IDX = [33, 160, 158, 133, 153, 144]
        self.RIGHT_EYE_IDX = [362, 385, 387, 263, 373, 380]

        self.EYE_AR_THRESH = ear_threshold
        self.eye_closed_start = None

    def _eye_aspect_ratio(self, lm, idx, w, h):
        p = [(lm[i].x * w, lm[i].y * h) for i in idx]
        A = math.hypot(p[1][0] - p[5][0], p[1][1] - p[5][1])
        B = math.hypot(p[2][0] - p[4][0], p[2][1] - p[4][1])
        C = math.hypot(p[0][0] - p[3][0], p[0][1] - p[3][1])
        return (A + B) / (2.0 * C)

    def detect(self, frame):
        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = self.face_mesh.process(frame_rgb)

        if not results.multi_face_landmarks:
            self.eye_closed_start = None
            return "no_face", 0.0

        lm = results.multi_face_landmarks[0].landmark
        h, w = frame.shape[:2]

        leftEAR = self._eye_aspect_ratio(lm, self.LEFT_EYE_IDX, w, h)
        rightEAR = self._eye_aspect_ratio(lm, self.RIGHT_EYE_IDX, w, h)

        if leftEAR < self.EYE_AR_THRESH and rightEAR < self.EYE_AR_THRESH:
            if self.eye_closed_start is None:
                self.eye_closed_start = time.time()
            return "closed", time.time() - self.eye_closed_start
        else:
            self.eye_closed_start = None
            return "open", 0.0
```

```
detector = EyeSleepDetector(ear_threshold=0.25)

cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
prev_time = time.time()
FRAME_SKIP = 6
count = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break

    if count % FRAME_SKIP == 0:
        state, closed_time = detector.detect(frame)
    count += 1

    # FPS
    curr_time = time.time()
    fps = 1.0 / (curr_time - prev_time)
    prev_time = curr_time

    print(f"FPS: {fps:.2f} | State: {state} | Closed: {closed_time:.2f}s")

    if cv2.waitKey(1) & 0xFF == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

5. Lane Detection

```
import cv2
import numpy as np
import time

def nothing(x):
    pass

class LaneDetector:
    def __init__(self):
        # --- Trackbar để điều chỉnh HSV threshold ---
        cv2.namedWindow("Trackbars")
        cv2.createTrackbar("L - H", "Trackbars", 0, 255, nothing)
        cv2.createTrackbar("L - S", "Trackbars", 0, 255, nothing)
        cv2.createTrackbar("L - V", "Trackbars", 113, 255, nothing)
        cv2.createTrackbar("U - H", "Trackbars", 255, 255, nothing)
        cv2.createTrackbar("U - S", "Trackbars", 43, 255, nothing)
        cv2.createTrackbar("U - V", "Trackbars", 255, 255, nothing)

        self.prevLx, self.prevRx = [], []

    def process_frame(self, frame):
```

```
        start_time = time.time()
        frame = cv2.resize(frame, (640, 480))

        # --- Điểm góc cho perspective transform ---
        tl, tr = (250, 350), (390, 350)
        bl, br = (80, 472), (560, 472)
        for pt in [tl, tr, bl, br]:
            cv2.circle(frame, pt, 5, (0,0,255), -1)

        # --- Perspective transform ---
        pts1 = np.float32([tl, bl, tr, br])
        pts2 = np.float32([[0,0],[0,480],[640,0],[640,480]])
        matrix = cv2.getPerspectiveTransform(pts1, pts2)
        transformed_frame = cv2.warpPerspective(frame, matrix, (640,480))

        # --- HSV thresholding ---
        hsv_frame = cv2.cvtColor(transformed_frame, cv2.COLOR_BGR2HSV)
        l_h = cv2.getTrackbarPos("L - H", "Trackbars")
        l_s = cv2.getTrackbarPos("L - S", "Trackbars")
        l_v = cv2.getTrackbarPos("L - V", "Trackbars")
        u_h = cv2.getTrackbarPos("U - H", "Trackbars")
        u_s = cv2.getTrackbarPos("U - S", "Trackbars")
        u_v = cv2.getTrackbarPos("U - V", "Trackbars")

        lower = np.array([l_h, l_s, l_v])
        upper = np.array([u_h, u_s, u_v])
        mask = cv2.inRange(hsv_frame, lower, upper)

        # --- Tự động điều chỉnh L-V ---
        white_ratio = cv2.countNonZero(mask) / mask.size
        adjust_speed = 2
        l_v = min(255, l_v + adjust_speed) if white_ratio > 0.10 else max(0, l_v -
adjust_speed) if white_ratio < 0.05 else l_v
        cv2.setTrackbarPos("L - V", "Trackbars", int(l_v))

        # --- Histogram để xác định lane bases ---
        histogram = np.sum(mask[mask.shape[0]//2:, :], axis=0)
        midpoint = histogram.shape[0]//2
        left_base = np.argmax(histogram[:midpoint])
        right_base = np.argmax(histogram[midpoint:]) + midpoint

        # --- Sliding window ---
        y = 472
        lx, rx = [], []
        msk = mask.copy()
        left_temp, right_temp = left_base, right_base

        while y > 0:
            # Left lane
            img = mask[y-40:y, left_temp-50:left_temp+50]
            contours, _ = cv2.findContours(img, cv2.RETR_TREE,
cv2.CHAIN_APPROX_SIMPLE)
            found = False
```

```
            for cnt in contours:
                M = cv2.moments(cnt)
                if M["m00"] != 0:
                    cx = int(M["m10"]/M["m00"])
                    lx.append(left_temp-50+cx)
                    left_temp = left_temp-50+cx
                    found = True
                    break
            if not found:
                lx.append(left_temp)

            # Right lane
            img = mask[y-40:y, right_temp-50:right_temp+50]
            contours, _ = cv2.findContours(img, cv2.RETR_TREE,
cv2.CHAIN_APPROX_SIMPLE)
            found = False
            for cnt in contours:
                M = cv2.moments(cnt)
                if M["m00"] != 0:
                    cx = int(M["m10"]/M["m00"])
                    rx.append(right_temp-50+cx)
                    right_temp = right_temp-50+cx
                    found = True
                    break
            if not found:
                rx.append(right_temp)

            y -= 40

        # --- Xử lý khi không tìm thấy ---
        lx = lx if len(lx)>0 else self.prevLx if self.prevLx else [left_base]*12
        rx = rx if len(rx)>0 else self.prevRx if self.prevRx else [right_base]*12
        self.prevLx, self.prevRx = lx, rx

        # --- Fit lane & tính offset, angle ---
        min_len = min(len(lx), len(rx))
        y_values = [472 - i*40 for i in range(min_len)]
        left_points = [(lx[i], y_values[i]) for i in range(min_len)]
        right_points = [(rx[i], y_values[i]) for i in range(min_len)]

        try:
            left_fit = np.polyfit([p[1] for p in left_points], [p[0] for p in
left_points], 2)
            right_fit = np.polyfit([p[1] for p in right_points], [p[0] for p in
right_points], 2)
            y_eval = 480
            left_curv =
((1+(2*left_fit[0]*y_eval+left_fit[1])**2)**1.5)/abs(2*left_fit[0])
            right_curv =
((1+(2*right_fit[0]*y_eval+right_fit[1])**2)**1.5)/abs(2*right_fit[0])
            curvature = (left_curv+right_curv)/2
            lane_center = (left_base + right_base)/2
            car_pos = 320
```

```
            lane_offset = (car_pos-lane_center)*3.7/640
            steering_angle = np.degrees(np.arctan(lane_offset/curvature))

            # Vẽ lane & overlay
            top_left = (lx[0], 472)
            bottom_left = (lx[min_len-1], y_values[min_len-1])
            top_right = (rx[0], 472)
            bottom_right = (rx[min_len-1], y_values[min_len-1])
            quad = np.array([top_left,bottom_left,bottom_right,top_right],
dtype=np.int32).reshape((-1,1,2))
            overlay = transformed_frame.copy()
            cv2.fillPoly(overlay,[quad],(0,255,0))
            cv2.addWeighted(overlay,0.2,transformed_frame,0.8,0,transformed_frame)

            inv_matrix = cv2.getPerspectiveTransform(pts2, pts1)
            lane_img = cv2.warpPerspective(transformed_frame, inv_matrix,
(640,480))
            result = cv2.addWeighted(frame,1,lane_img,0.5,0)

            # Draw center line
            end_x = int(320 + 100*np.sin(np.radians(steering_angle)))
            end_y = int(480 - 100*np.cos(np.radians(steering_angle)))
            cv2.line(result,(320,480),(end_x,end_y),(255,0,0),2)

            cv2.putText(result,f'Offset: {lane_offset:.2f}
m',(30,70),cv2.FONT_HERSHEY_SIMPLEX,1,(255,255,255),2)
            if abs(lane_offset)>0.7:

cv2.putText(result,'WARNING',(30,240),cv2.FONT_HERSHEY_SIMPLEX,1,(0,0,255),2)
            cv2.putText(result,f'Angle: {steering_angle:.2f}
deg',(30,110),cv2.FONT_HERSHEY_SIMPLEX,1,(255,255,255),2)
            cv2.putText(result,'Lane
detected',(30,150),cv2.FONT_HERSHEY_SIMPLEX,1,(0,255,0),2)
        except:
            result = frame.copy()
            cv2.putText(result,'Lane detection
error',(30,30),cv2.FONT_HERSHEY_SIMPLEX,1,(0,0,255),2)

        # --- Hiển thị mask ---
        mask_display = cv2.cvtColor(mask,cv2.COLOR_GRAY2BGR)
        # --- Combine views ---
        top_row = np.hstack((cv2.resize(frame,(320,240)),
cv2.resize(transformed_frame,(320,240))))
        bottom_row = np.hstack((cv2.resize(mask_display,(320,240)),
cv2.resize(result,(320,240))))
        combined = np.vstack((top_row,bottom_row))

        fps = 1.0/(time.time()-start_time)
        cv2.putText(combined,f"FPS:
{fps:.2f}",(20,20),cv2.FONT_HERSHEY_SIMPLEX,0.5,(0,0,255),2)

        return combined
```

6. Lane Detection(inference-only)

```
import cv2
import numpy as np
import time


class LaneDetectorFPSOnly:
    def __init__(self):
        # Cố định HSV (không dùng trackbar)
        self.lower = np.array([0, 0, 113])
        self.upper = np.array([255, 43, 255])

        self.prevLx, self.prevRx = [], []

        # Perspective points
        self.tl, self.tr = (250, 350), (390, 350)
        self.bl, self.br = (80, 472), (560, 472)

        self.pts1 = np.float32([self.tl, self.bl, self.tr, self.br])
        self.pts2 = np.float32([[0, 0], [0, 480], [640, 0], [640, 480]])

        self.matrix = cv2.getPerspectiveTransform(self.pts1, self.pts2)

    def process_frame(self, frame):
        start_time = time.time()

        # Perspective transform
        warped = cv2.warpPerspective(frame, self.matrix, (640, 480))

        # HSV threshold
        hsv = cv2.cvtColor(warped, cv2.COLOR_BGR2HSV)
        mask = cv2.inRange(hsv, self.lower, self.upper)

        # Histogram
        histogram = np.sum(mask[mask.shape[0] // 2:, :], axis=0)
        midpoint = histogram.shape[0] // 2
        left_base = np.argmax(histogram[:midpoint])
        right_base = np.argmax(histogram[midpoint:]) + midpoint

        # Sliding window
        y = 472
        lx, rx = [], []
        left_temp, right_temp = left_base, right_base

        while y > 0:
            # Left
            roi = mask[y - 40:y, left_temp - 50:left_temp + 50]
            contours, _ = cv2.findContours(roi, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
            if contours:
                M = cv2.moments(contours[0])
                if M["m00"] != 0:
```

```
                cx = int(M["m10"] / M["m00"])
                left_temp = left_temp - 50 + cx
            lx.append(left_temp)

            # Right
            roi = mask[y - 40:y, right_temp - 50:right_temp + 50]
            contours, _ = cv2.findContours(roi, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
            if contours:
                M = cv2.moments(contours[0])
                if M["m00"] != 0:
                    cx = int(M["m10"] / M["m00"])
                    right_temp = right_temp - 50 + cx
            rx.append(right_temp)

            y -= 40

        # Fallback
        lx = lx if lx else self.prevLx
        rx = rx if rx else self.prevRx
        self.prevLx, self.prevRx = lx, rx

        # Fit lane (chỉ để giữ pipeline, không vẽ)
        if lx and rx:
            min_len = min(len(lx), len(rx))
            y_vals = [472 - i * 40 for i in range(min_len)]
            np.polyfit(y_vals, lx[:min_len], 2)
            np.polyfit(y_vals, rx[:min_len], 2)

        fps = 1.0 / (time.time() - start_time)
        return fps

import cv2

detector = LaneDetectorFPSOnly()

cap = cv2.VideoCapture("VID/VN_1.mp4")
fps_sum = 0
count = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break

    fps = detector.process_frame(frame)

    fps_sum += fps
    count += 1

    print(f"FPS: {fps:.2f}")

    if cv2.waitKey(1) & 0xFF == 27:
```

```
        break

print(f"\nFPS trung bình: {fps_sum / max(count,1):.2f}")

cap.release()
cv2.destroyAllWindows()
```

7. Main

```
from multiprocessing import Process, Queue, set_start_method
import cv2
import numpy as np
from eyes_detection import EyeSleepDetector  # import class bạn đã viết
from object_detection import ObjectDetector
from lane_detection import LaneDetector
import winsound


def camera_reader_video(video_path, queue1,queue2):
    cap = cv2.VideoCapture(video_path)
    while True:
        ret, frame = cap.read()
        if not ret:
            queue1.put(None)
            queue2.put(None)
            break
        if not queue1.full() and not queue2.full():
            queue1.put(frame)
            queue2.put(frame)
    cap.release()

def camera_reader(video_path, queue):
    cap = cv2.VideoCapture(0)
    while True:
        ret, frame = cap.read()
        if not ret:
            queue.put(None)  # báo hiệu hết video
            break
        if not queue.full():
            queue.put(frame)
    cap.release()

def run_object_detection(queue):

    weights = "H:/PBL5/yolov3_tiny/yolov3-tiny.weights"
    cfg = "H:/PBL5/yolov3_tiny/yolov3-tiny.cfg"

    net = cv2.dnn.readNetFromDarknet(cfg, weights)
    detector = ObjectDetector(net)

    # Hình thang
    tl = (250, 350)
    tr = (390, 350)
    bl = (80, 472)
    br = (560, 472)
```

```
    trap = np.array([tl, tr, br, bl], dtype=np.int32)

    while True:
        frame = queue.get()
        if frame is None:
            break

        frame = cv2.resize(frame, (640, 480))

        # Detect
        output, fps, boxes = detector.detect_objects(frame)

        # Vẽ hình thang
        cv2.polylines(output, [trap], True, (255,0,0), 2)

        # Kiểm tra object trong vùng
        for (x, y, w, h) in boxes:
            cx = x + w//2
            cy = y + h//2

            # vẽ tâm object
            cv2.circle(output, (cx,cy), 5, (0,255,255), -1)

            inside = cv2.pointPolygonTest(trap, (cx,cy), False)
            if inside >= 0:
                cv2.putText(output, "CANH BAO: Object trong vung!", (10,80),
                            cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255), 2)
        cv2.imshow("Object Detection", output)
        if cv2.waitKey(1) == 27:
            break

    cv2.destroyAllWindows()



def run_lane_detection(queue):
    detector = LaneDetector()  # Tạo object xử lý lane
    cv2.namedWindow("Lane Detection", cv2.WINDOW_NORMAL)

    while True:
        frame = queue.get()
        if frame is None:
            print("[INFO] Lane detection stopped.")
            break

        frame = cv2.resize(frame, (640, 480))
        lane_frame = detector.process_frame(frame)

        cv2.imshow("Lane Detection", lane_frame)

        if cv2.waitKey(1) & 0xFF == 27:  # ESC
            break
```

```
    cv2.destroyAllWindows()

def run_sleep_detection(queue):
    detector = EyeSleepDetector()
    while True:
        frame = queue.get()
        if frame is None:
            break
        eye_state, duration = detector.detect_sleepiness(frame)
        print(f"Eye state: {eye_state}, duration: {duration:.2f}s")
        # Có thể hiển thị frame nếu muốn
        cv2.putText(frame, f"{eye_state}: {duration:.2f}s", (30,50),
                    cv2.FONT_HERSHEY_SIMPLEX, 1, (0,0,255) if eye_state=="closed"
else (0,255,0), 2)
        cv2.imshow("Sleep Detection", frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break
    cv2.destroyAllWindows()

# --- Gọi hàm ---
if __name__ == "__main__":
    set_start_method("spawn")  # Bắt buộc cho Windows
    frame_queue = Queue(maxsize=1)
    frame_queue2 = Queue(maxsize=1)
    frame_queue1 = Queue(maxsize=1)

    p1 = Process(target=camera_reader, args=("0", frame_queue))
    p2 = Process(target=run_sleep_detection, args=(frame_queue,))
    p3 = Process(target=camera_reader_video, args=("VID/VN_4.mp4",frame_queue2,
frame_queue1))
    p4 = Process(target=run_object_detection, args=(frame_queue2,))
    p5 = Process(target=run_lane_detection, args=(frame_queue1,))

    p1.start()
    p2.start()
    p3.start()
    p4.start()
    p5.start()

    p1.join()
    p2.join()
    p3.join()
    p4.join()
    p5.join()
```